Cheatsheet

Libero Biagi

June 4, 2026

Intro

This is a general cheatsheet where I will try to put all the coding, the formulas, the terminology and the possible exercises that might be useful for the exams.

Contents

1 Computational intelligence **4**
1.1 Introduction . . . 4
1.2 Optimization algorithms . . . 5
1.3 Fitness landscape . . . 6
1.4 Annealing . . . 8
1.5 Practical applications of Hill climbing . . . 8
1.6 Simulated annealing . . . 9
1.7 Genetic algorithms . . . 11
1.8 Again Genetic algorithms . . . 12
1.8.1 Fitness proportionate selection . . . 13
1.8.2 Ranking selection . . . 13
1.8.3 Tournament selection . . . 13
1.9 Crossover and mutation . . . 14
1.9.1 Crossover . . . 14
1.9.2 Mutation . . . 14
1.9.3 General function of genetic algorithms . . . 14
1.9.4 Hyperparameters of genetic algorithms . . . 15
1.10 Why genetic algorithms work . . . 15
1.10.1 Effect of selection on schemata . . . 16
1.10.2 Effect of crossover on schemata . . . 16
1.10.3 Combined effect of selection and crossover . . . 17
1.10.4 Effect of mutation . . . 17
1.10.5 Building blocks' hypothesis . . . 17
1.10.6 K-armed bandit problem . . . 18
1.11 Problems with genetic algorithms . . . 18
1.11.1 Premature convergence . . . 18
1.11.2 Position problem . . . 20
1.12 Multi-objective optimization . . . 21
1.13 Continuous optimization . . . 22
1.13.1 Geometric crossover . . . 22
1.13.2 Geometric mutation . . . 23
1.14 Particle Swarm Optimization . . . 23

2 Big data analytics **24**
2.1 Introduction . . . 24
2.1.1 What is Big data . . . 24
2.1.2 Data sources . . . 24
2.1.3 The Vs of Big data . . . 25
2.1.4 The Big data ecosystem . . . 25
2.1.5 Applications of Big data . . . 25
2.2 Distributed storage, ecosystems and programming paradigms . . . 26
2.2.1 Distributed storage and file systems . . . 26
2.2.2 Distributed computing models . . . 26

2.2.3 The big data ecosystem . . . 27
2.2.4 Programming paradigms . . . 27
2.3 Distributed parallel processing and the map reduce paradigm . . . 27
2.3.1 Distributed parallel processing . . . 27
2.3.2 The MapReduce paradigm . . . 28
2.3.3 Case study . . . 28
2.4 The Hadoop ecosystem . . . 28
2.4.1 Hadoop . . . 28
2.4.2 Filesystems and HDFS basics . . . 29
2.4.3 Hadoop's execution model . . . 29
2.4.4 Optimization and refinements . . . 30
2.4.5 YARN and scheduling . . . 30
2.4.6 Hive, Pig and Sqoop . . . 31
2.4.7 Summary . . . 32
2.5 Introduction to Spark . . . 32
2.6 DataFrames in Spark and Spark-Hadoop relationship . . . 34
2.6.1 Recap . . . 34
2.6.2 DataFrames . . . 34
2.6.3 Datasets . . . 36
2.6.4 Spark-Hadoop relationship . . . 36
2.7 Spark in Cloud-based computational environments . . . 36
2.7.1 Databricks . . . 37
2.7.2 AWS Elastic MapReduce (EMR) . . . 37
2.7.3 Google Cloud Dataproc . . . 38
2.7.4 Azure HDInsight . . . 38
2.7.5 Kubernetes . . . 38
2.7.6 Google Colab . . . 38
2.7.7 Lightning AI . . . 39
2.8 Performance and data engineering in Spark . . . 39
2.8.1 Why Spark jobs are slow . . . 39
2.8.2 How Spark optimizes queries . . . 39
2.8.3 Data engineering for performance . . . 39
2.9 Big data storage technologies and relation with Spark . . . 41
2.9.1 Intro to big data storage . . . 41
2.9.2 Key-value DB . . . 41
2.9.3 Document DB - MongoDB focus . . . 42
2.9.4 Wide column DB . . . 43
2.9.5 Graph DB . . . 43
2.9.6 Other DB types . . . 43
2.10 Machine Learning in Spark . . . 43
2.10.1 Introduction . . . 43
2.10.2 Spark MLlib concepts . . . 44
2.10.3 Algorithms . . . 44
2.10.4 Evaluation . . . 45
2.10.5 Advanced Topics . . . 45
2.11 Pipelines in Spark . . . 45
2.11.1 Introduction . . . 45
2.11.2 Spark pipeline API . . . 45
2.11.3 Feature engineering pipelines . . . 46
2.11.4 Model training and evaluation . . . 46
2.11.5 Hyperparameter tuning . . . 46
2.12 Graph Analytics . . . 46
2.12.1 Introduction . . . 46
2.12.2 Spark GraphX . . . 46
2.12.3 GraphFrames . . . 47
2.12.4 Advanced topics . . . 47
2.13 Streaming . . . 47
2.13.1 Introduction . . . 47
2.13.2 Structured streaming overview . . . 47
2.13.3 Streaming concepts . . . 48
2.13.4 Spark structured streaming API . . . 48

2.13.5 Advanced streaming . . . 49
2.13.6 Challenges and best practices . . . 49

3 Neural and evolutionary learning 50
3.1 Machine learning review . . . 50
3.1.1 Performance measures for regression . . . 50
3.1.2 Performance measures for classification . . . 51
3.1.3 Features . . . 53
3.1.4 Experimental ML study . . . 53
3.2 Genetic programming . . . 53
3.2.1 Evolutionary algorithms . . . 53
3.2.2 Symbolic regression . . . 54
3.2.3 Why do evolutionary algorithms work? . . . 54
3.2.4 Peculiarities of genetic programming . . . 54
3.2.5 Weak points of genetic programming . . . 55
3.3 Geometric semantic genetic programming . . . 56
3.3.1 Optimization problems . . . 56
3.3.2 Fitness Landscapes . . . 56
3.3.3 Genetic algorithms . . . 56
3.3.4 Genetic programming . . . 57
3.3.5 Implementation of geometric semantic operators . . . 57
3.3.6 Discussion . . . 57
3.3.7 Open issues . . . 57
3.4 Semantic Learning algorithm based on inflate and deflate mutations . . . 58
3.5 Neural networks . . . 59
3.6 Introduction . . . 59
3.6.1 Perceptron . . . 59
3.6.2 ADALINE . . . 60
3.6.3 Non-linearly separable problems . . . 61
3.6.4 Backpropagation . . . 62
3.6.5 Cyclic (or Recursive) Neural networks . . . 63
3.7 Neuroevolution and NEAT . . . 63

4 Text mining 64
4.1 Introduction to NLP . . . 64
4.1.1 Challenges of natural language . . . 64
4.1.2 The NLP pipeline . . . 64
4.1.3 Text preprocessing methods . . . 64
4.2 Word representation . . . 65
4.2.1 Classification with BoW . . . 65
4.2.2 Word representations . . . 65
4.2.3 Vector embeddings (Word2Vec) . . . 66
4.2.4 Classification with Word2Vec . . . 66
4.3 Sequential models . . . 66
4.3.1 Review of word embeddings . . . 66
4.3.2 RNNs . . . 66
4.3.3 LSTM . . . 67
4.3.4 Seq-to-seq models . . . 68
4.4 Attention and transformers . . . 69
4.4.1 Attention mechanisms . . . 69
4.4.2 Transformer architecture . . . 70
4.4.3 Transformer encoders . . . 70
4.5 LLMs . . . 71
4.5.1 Transformer architecture . . . 71
4.5.2 Transform encoders . . . 72
4.5.3 Classification with encoders . . . 73
4.5.4 Self-attention . . . 73
4.5.5 Deeper into the encoder block . . . 74
4.5.6 The architecture of decoders . . . 74

1 Computational intelligence

1.1 Introduction

Definitions to start the course

- **Problem** →Task to be solved automatically (find the maximum number in a list)
- **Algorithm** →Process to solve a problem, set of precise and not ambiguous actions that all together will allow us to reach the solution
- **Program** →Translation of an algorithm into a programming language

Motivation

- In classical computational methods we start from a problem, and we have a human being that imagines an algorithm (problem-solving, it's creative and informal). Then software engineers will put the ideas into a computer in order to get a program (implementation, it's automatic and not creative).
- We need computational intelligence because traditional methods in many cases fail, usually on the first step. We can have many problems where imaging solutions or algorithms it's either impossible or very hard for human being to imagine a solution. This is the case of **complex problems**

Some examples of complex problems

- Categorization of clients
- Face recognition
- Find a new drug to cure a disease
- Driving a robot in a 3-D space

Our idea is to give computers or informatic systems the ability to learn autonomously how to solve a problem. We define **learning** as improving something by means of experience. Those computers can also understand that the generated solution in not good enough, and so they should understand a better way to find the optimal solution.

One of the most famous ways to improve those computers are bio-inspired algorithms. Like the **neural networks** (took inspiration from the brain), **evolutionary computation** (theory of evolution from Darwin, considering evolution as a sort of learning) where we have genetic algorithms, differential evolution or genetic programming, **swarm intelligence based algorithms** (system of animals are better at solve problems with respect to singular entities) like particle swarm optimization, **fuzzy systems** and **local search** like hill climbing simulated annealing.

One category of complex problems are the so-called optimization problems. Even though they are complex they are easy to comprehend and to analyze.

Informally solving an optimization problem means to find the best solution(s) in a typically huge set of possible alternative solutions. We are able to find if something is a solution and to evaluate the quality of the solution with respect to the others. More formally an optimization problem is a pair of objects S and f where

- S →set of all existing solutions (search space)
- f →function that evaluates the solutions (fitness or cost or quality function)

An optimization problem can be:

$$\textbf{Maximization:} \quad \text{find } x^* \in S \text{ such that } f(x^*) \geq f(y) \quad \forall y \in S.$$

$$\textbf{Minimization:} \quad \text{find } x^* \in S \text{ such that } f(x^*) \leq f(y) \quad \forall y \in S.$$

To solve this kind of problems we can use an optimization algorithm, a machine that is able to output multiple solutions (iterative machine).

No free lunch theorem →the average performance of all optimization algorithms calculated on all existing optimization problems is the same. We care about the probability of finding the best solution and the time to reach it. We can't have one algorithm for every problem then we need to find a solution for every problem.

1.2 Optimization algorithms

Optimization problem →a pair (S, f) where

- S is the set of all solutions (search space)
- f is the function (fitness function) where f : S →$\mathbb{R}$

An optimization problem can be:

$$\textbf{Maximization:} \quad \text{find } x^* \in S \text{ such that } f(x^*) \geq f(y) \quad \forall y \in S.$$

$$\textbf{Minimization:} \quad \text{find } x^* \in S \text{ such that } f(x^*) \leq f(y) \quad \forall y \in S.$$

x can be either a global optimum or a local one, we want to find the global(s) optima.

To find the solution we will use some optimization algorithm, by definition an optimization algorithm is an iterative method that at the end of each iteration returns a solution ∈ S. One of the usual algorithm is **Random search**.
As already seen we have the no free lunch theorem that stops us from finding a perfect solution.

No Free Lunch Theorem →Let A_1 and A_2 be two optimization algorithms. When performance is averaged uniformly over all possible objective functions, A_1 and A_2 have identical expected performance.

The consequences of this theorem are that

1. It is impossible to have an algorithm that is better than all the others and on all possible problems (no super algorithm can exist).
2. Given a problem it can't exist an automatic method to find the best algorithm. Then we need to test, experiment and use our prior knowledge.

Now we will see some algorithms. We start with **Hill climbing**. Before anything we need to clarify that we can't find every possible solution, so we need to use certain starting points.

Neighborhood. Let $N : S \to 2^S$ be a neighborhood function that maps a solution $x \in S$ to a set of neighboring solutions $N(x) \subseteq S$.
Starting from an initial solution x_0 (typically chosen at random), the objective function is evaluated over $N(x_k)$, and the best neighbor is selected:

$$x_{k+1} = \arg \min_{y \in N(x_k)} f(y)$$

(for a minimization problem).
This process is repeated iteratively until a stopping criterion is met.

Example

- S = students in the room
- f = student number
- Maximization problem
- Neighborhood = the students around the selected student

Local optimum →given an optimization problem (S, f) a local optimum is a solution x such that $\forall y \in N(x), f(x)$ is better than $f(y)$. By definition hill climbing will return a local optimum with no guarantee that it will also be global. We can find the global optimum by expanding the radius or by trying more initialization

In pseudocode (general case)

1. Initialize the current solution x (typically at random).
2. Repeat

2.1 Generate a neighbor y of x.

2.2 If $f(y)$ is better than or equal to $f(x)$, set $x := y$.

Until for all neighbors y of x, $f(y)$ is worse than $f(x)$.

3. Return x.

In strict hill climbing we change the point only if the fitness is strictly better than the previous. With this setting we can have an infinite iteration between 2 equal points. We can set a timeout if we see the same points n times.

Taboo search can help the search by using memory, avoid the already checked points.

Example

- $S = \{i \in \mathbb{N} : 0 \leq i \leq 15\}$
- f(i) number of 1s in the binary code of i
- Maximization
- Neighborhood = i and j $\iff$ |i - j| = 1

We produce a plot

- X →all solutions produced ordered by their neighborhood
- Y →all the fitness values

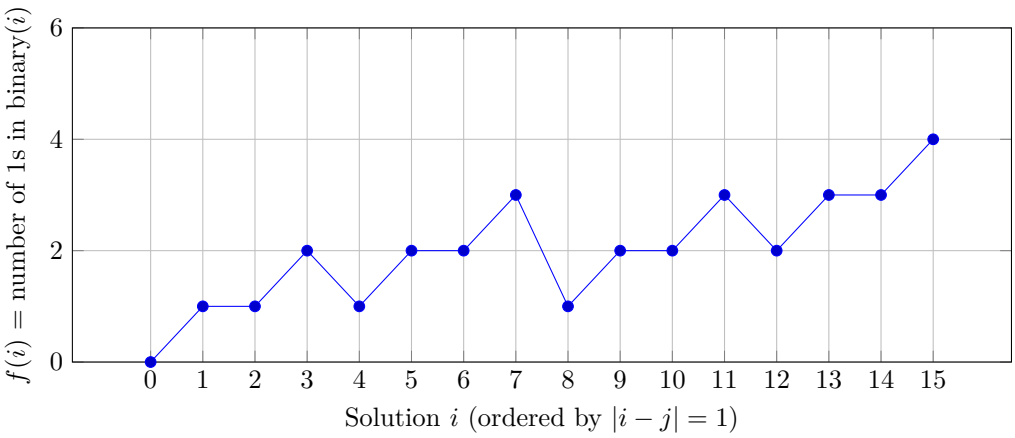


Figure 1: Fitness landscape for $S = \{0, \ldots, 15\}$ with neighborhood $|i - j| = 1$.

With the fitness landscape we can find the difficulty of the problem. A very smooth one is usually of a simple problem while a very oscillating one will be probably more difficult.

1.3 Fitness landscape

In the last class we saw what is a fitness landscape. The plot in the 2-D setting is obtained by plotting on the axis

- Horizontal →All individual in the search space sorted according to the neighborhood
- Vertical →the fitness

Given an optimization problem (S, f) and a neighborhood N, a solution $x \in S$ is called a local optimum if $\forall y \in N(x)$ f(y) is worse than f(x). By construction hill climbing will return a local optimum with no guarantee that it will also be a global optimum.

The fitness landscape gives a visual idea of how easy/hard is for an algorithm to solve the problem

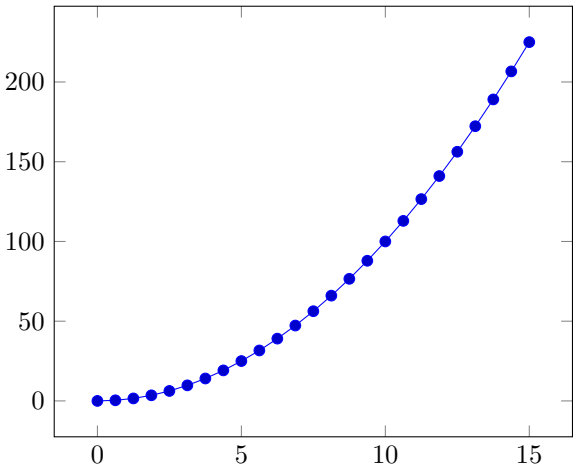
- Smooth →easy

- Rugged →hard

But in practice we can't draw the landscape because the search space is humongous, and the neighborhood is usually multidimensional.

Example

- $S = \{i \in \mathbb{N} | 0 \leq i \leq 15\}$
- f(i) = i^2
- N = The solutions x and y are neighbors $\iff |x - y| = 1$
- Maximization



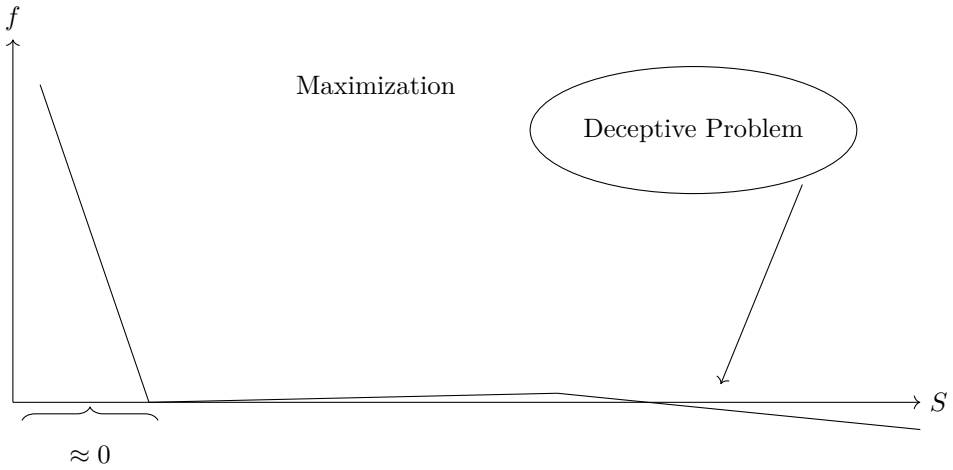
Here hill climbing will arrive at the global optimum.

If we want to look for the number of 1 in the binary code of i we can encounter problems and stay stuck in a local optimum without being able to find the global one.

The definition of neighborhood is not fixed, we can define it in 2 ways.

- N(x) →{ $y \in s | d(x, y) \leq k$} →distance based
- N(x) →{ $y \in s | y = op(x)$} →action based

Example



Random search is "good" for this type of problem.

Hill climbing pros

- Simple
- Efficient
- Versatile

Cons

- Always stops on local optima

How to improve hill climbing

- Parallel hill climbing
- Enlarge the neighborhood
- Give the possibility of worsening the fitness of the current solution (within a given probability)

1.4 Annealing

We now introduce the concept of annealing →an experiment aimed at finding the solid state of a material with the minimum level of energy.

Metropolis algorithm

1. Consider the initial state of the material with energy E_i
2. Alter the state to get a new energy E_j
3. If $E_j \leq E_i$ accept the new state and continue
4. Else accept the state with probability $exp(-\frac{|E_i - E_j|}{K_B T})$

Where

- T = Temperature
- K_B = Boltzmann constant

Simulated annealing

- Current solution = x
- Neighbor = y

$$P(y) = \begin{cases} 1 \text{ if f(y) better or equal than f(x)} \\ exp(-\frac{|f(x) - f(y)}{C}) \text{ otherwise} \end{cases}$$

Where C is a control parameter

1.5 Practical applications of Hill climbing

Test case 1 →Given a set of possible investments (N), for each investment we know

- Cost
- Estimated gain

We also know the budget of the client. What subset of the investments should our client do?
Such that

- The total cost is smaller or equal to the budget
- The profit is maximized

Using a numeric example

- N = 10
- Costs = {23, 31, 29, 44, 53, 38, 63, 85, 89, 82 }
- Profits = {92, 57, 49, 68, 60, 43, 67, 84, 87, 72 }
- Budget = 165

Questions to answer

1. What are the solutions ?
2. How it is convenient to represent the solutions?
3. What's the fitness?
4. If the algorithm is neighborhood based, then what is the neighborhood?

Golden rule →**DO NOT solve the problem**
Answers

1. All the subsets of the set of investments
2. As sequences of values, a string of bits of length N. The array will have a pair of cost and profit. We use 0s and 1s to "1-hot encoding" them.
3. The sum for all the indexes where x is 1 of all the profits. $\sum_{i:x[i]=1} p[i]$. We have a maximization problem. We only care about solutions that have a cost less than the budget. The fitness if this condition is not accepted we have -1 or 0 as fitness. Another way to define a non-good fitness is to use the negative sum of the costs such that the distance ourselves as much as possible from a bad solution.
4. The neighborhood is obtained here using the bit flip operator.

Test case 2 →Travel-sale-person: given N cities for which we know all the pairwise distances between themselves. Give one of these cities there is one specific one that is called home. We want the cyclic path that starts from home, visits all the cities once and only once and terminates at home using the shortest possible path.

We start from a matrix

- N = 5 →the matrix is 5x5
- Inside the matrix we have the costs/distances

Answers

1. Solutions →All cyclic paths starting and finishing in home and visiting all cities once.
2. Representation →An ordered string of the cities
3. Fitness →the sum of the distances in the string
4. Neighborhood →We can use a swapping operator

1.6 Simulated annealing

Simulated annealing is similar to hill climbing, but we add the concept of probability. This can lead to a worse fitness. The idea is that we can explore more possible solutions and obtain a better optimum. But we can also miss some of those. Sometimes can be convenient that our current solution is a random neighbor.

We can start by defining our current solution x and our candidate neighbor y.
The probability $P(y)$ is defined as:

$$P(y) = \begin{cases} 1 & \text{if } f(y) \leq f(x) \\ \exp\left(-\frac{|f(x)-f(y)|}{C}\right) & \text{otherwise} \end{cases}$$

Where C is a control parameter $C > 0$.

Afterward, we look for a random number in the interval $[0, 1]$ and if this number is $< P(y)$, we accept the move.

We start with a large C, and we decrease it during the algorithm. C will tend to 0 without reaching it, following the cooling schedule:

$$C = \frac{C}{h}$$

Simulated annealing follows the following structure

1. Initialize the current solution x (typically at random)
2. Initialize C and L
3. Repeat the following until termination
   - Repeat L times
     - Create a random neighbor x of y
     - Accept or not y with a certain probability
   - Update C (not mandatory update L)
4. Return x (or the best solution so far)

We terminate when we have found a certain condition that can be either a good enough solution or we have run a prefixed maximum number of iterations of the external loop.

Example

- S = i $\in \mathbb{N} | 0 \leq i \leq 15$
- $\forall i \in S, f(i)$ = number of 1s in the binary code of i
- Maximization
- Neighborhood →x and y are neighbors $\iff |x - y| = 1$

Step 1: Initialization
$x = 5$ (random initialization) $\rightarrow bin(5) = 101$
$f(x) = 2$
$N(x) = \{4, 6\}$
Assume $y = 6$ (random neighbor) $\rightarrow bin(6) = 110$
$f(y) = 2$
Since $f(y) \leq f(x)$, we accept the move: $x = 6$.

Step 2: Iteration
$x = 6$
$N(x) = \{5, 7\}$
Assume $y = 7$ (random neighbor) $\rightarrow bin(7) = 111$
$f(y) = 3$
Since $f(y) > f(x)$, we calculate the acceptance probability.

Step 3: Probability Calculation
Assume in this phase the control parameter is $C = 1$.
Current state: $f(x) = 3$
Candidate neighbor: $f(y) = 1$

$$P(acc\ y) = \exp\left(-\frac{|3-1|}{1}\right) = e^{-2} \simeq 0.13$$

This value represents a *"large" probability* for this phase of the algorithm.

Theorem of asymptotic convergence of simulated annealing

Given an optimization problem (S, f) and an appropriate neighborhood, if we decrease C so that it tends to 0 (without reaching it):
As $t \to \infty$ and $C \to 0$, we analyze the convergence:

$$\lim_{C \to 0} q_i(C) = \frac{1}{|S_{opt}|} \cdot g(i)$$

Where:

- $q_i(C)$ is the probability that Simulated Annealing (SA) will stabilize on solution i with control parameter C.
- S_{opt} is the set of global optima.

- $g(i)$ is an indicator function defined as:

$$g(i) = \begin{cases} 1 & \text{if } i \in S_{opt} \\ 0 & \text{otherwise} \end{cases}$$

This limit shows that as the temperature C approaches zero, the probability of being in a state i becomes uniform over the set of global optimal solutions and zero elsewhere. To be sure to find a global optimum we should execute the process a very large number of times.

The parameters L and C are respectively Large and slow (decreasing)

1.7 Genetic algorithms

Genetic algorithms share similarities with simulated annealing

- Both are optimization algorithms
- Both allow to look for good solutions without an exhaustive analysis of the search space
- Both start from random solutions and improve their quality iteratively

The differences on the other hand are

- The solutions (individuals) that are considered at a time are more than 1 (population)
- Inspired by the theory of evolution of Charles Darwin
- Individuals must be represented by strings of constant length

The theory of evolution has the following key elements

- Reproduction
- Ability of adaptation
- Inheritance
- Variation
- Competition

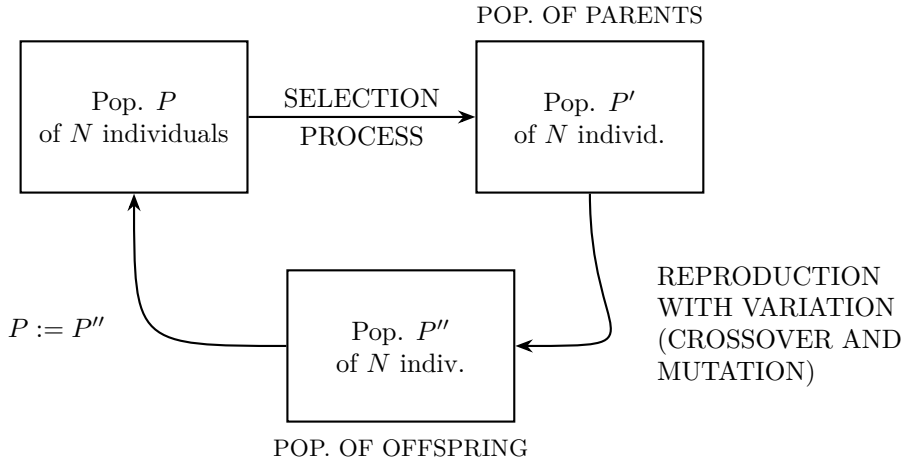
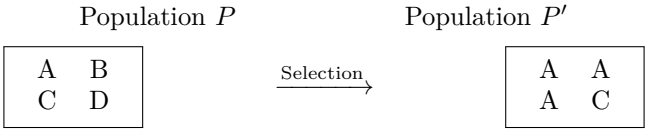


Figure 2: Genetic Algorithms Cycle

The **Selection Process** maps the initial population P to the intermediate population P' (parents):



In this example, individual **A** is selected three times due to its high fitness, **C** is selected once, while **B** and **D** are discarded.

The selection process is made of iterations of N independent executions of a selection algorithm. The selection algorithm chooses 1 solution from a population P.
The algorithm has the following general principles

1. It's probabilistic
2. Given any pair of independent A and B in P if f(A) is better than f(B) then A has a bigger probability than B of being selected
3. No individual in P has a probability equal to 0 of being chosen

Fitness Proportionate (Roulette Wheel) Selection
Given N individuals in population $P = \{1, 2, \ldots, N\}$, for each $i = 1, 2, \ldots, N$ we define the probability of selection as:

$$P(sel\ i) = \frac{f_i}{\sum_{j \in P} f_j}$$

Mechanism ($N = 4$):
This method acts like a roulette wheel where the area of each slice i is proportional to its fitness f_i. Individuals with higher fitness occupy a larger portion of the wheel, increasing their chances of being selected when the wheel is "spun" (represented by a random number $U \in [0, 1]$).

- i_1, i_2, i_3, i_4: Individuals in the population.
- U: Random pointer used to select the individual.

Other ways that can be used are **Ranking** and **Tournament** selection.

1.8 Again Genetic algorithms

We forgor something about simulated annealing. The theorem that we talked about miss something.
Basically we can only use simulated annealing in an appropriate algorithm, but we didn't define what we meant by appropriate.

From the part of asymptotic convergence. We must have a certain property →given any pair of solutions x and y it must be possible to build a sequence of solutions $a_1, a_2, ..., a_n$ such that $\forall i = 1, 2, 3, ..., n - 1, a_i$ and $a_{i+1}$are neighbors. So $a_1 = x$ and $a_n = y$. We say that the neighborhood is completely connected.

Now going back to genetic algorithms.
We start with a population P of N individuals. N is a hyperparameter. We modify the population with two steps.

- Selection phase →We have a population P' of N individuals
- Reproduction with Variation phase →We have a population P" of N individuals

The new current population will be P" and we repeat the two phases until we reach a stopping criterion, and we select as solution the best individual in the final P.
We create new individuals (offspring) via means of crossover and mutation.

The idea behind this algorithm comes from the Darwinian ideas of adaptation, competition, reproductions, inheritance and variation. The first two are for the first phase, while the other 3 for the second one. The selection phase is an iteration of N independent executions of a selection algorithm where the objective is to choose one solution from a population of N solutions.

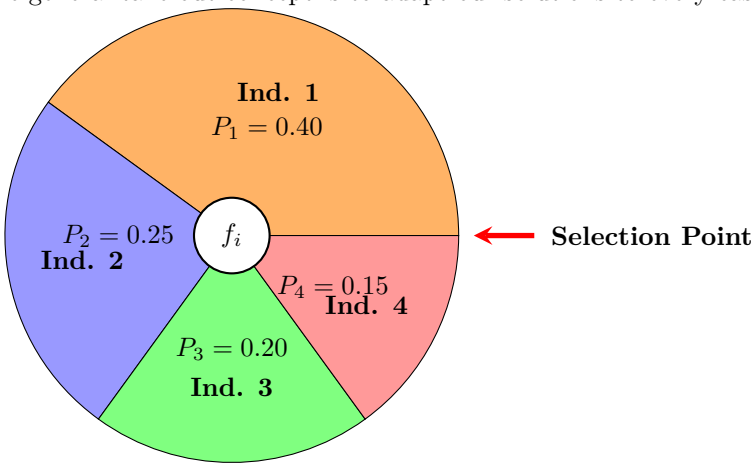
Is important that the selection algorithm is probabilistic since we want to have different solutions every time. The probability of selecting a solution must be based on fitness. All individuals must have a non-zero probability of being selected.

In this course we will talk about 3 main selection algorithms

1.8.1 Fitness proportionate selection

Also known as roulette wheel. We have a population of N individuals called 1, 2,..., n. For each i in 1, 2,..., n we have that the probability of selecting i we have $P(i) = \frac{f(i)}{\sum_{j \in P} f(j)}$. This is a maximization only algorithm, for minimization we can do 1 divided by fitness but if we have 0 fitness we use to add a constant to every possible value.

The general take out concept is to adapt our solutions to every case.

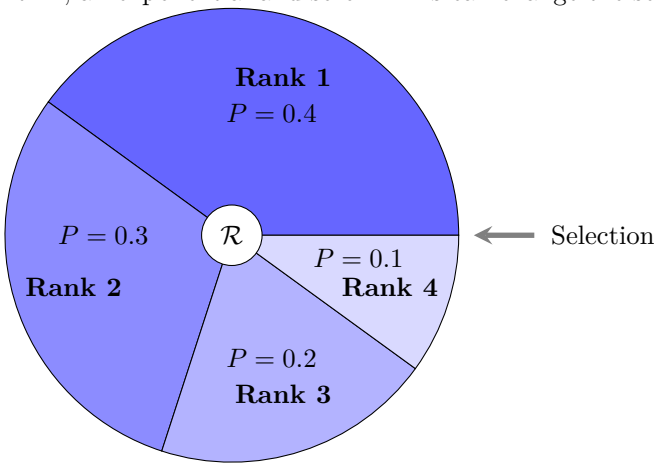


1.8.2 Ranking selection

We follow this procedure

1. Sort the individuals from the worst to the best based on fitness.
2. We calculate the probability selection based on the indexes. So $P(i) = \frac{\text{index}}{\sum \text{indexes}}$

This algorithm is not sensitive to fitness differences, we don't care about how much a certain individual is better than another one. We can generalize this algorithm by changing the function (now is linear) to have a logarithm, an exponential and so on. This can change the selection pressure.

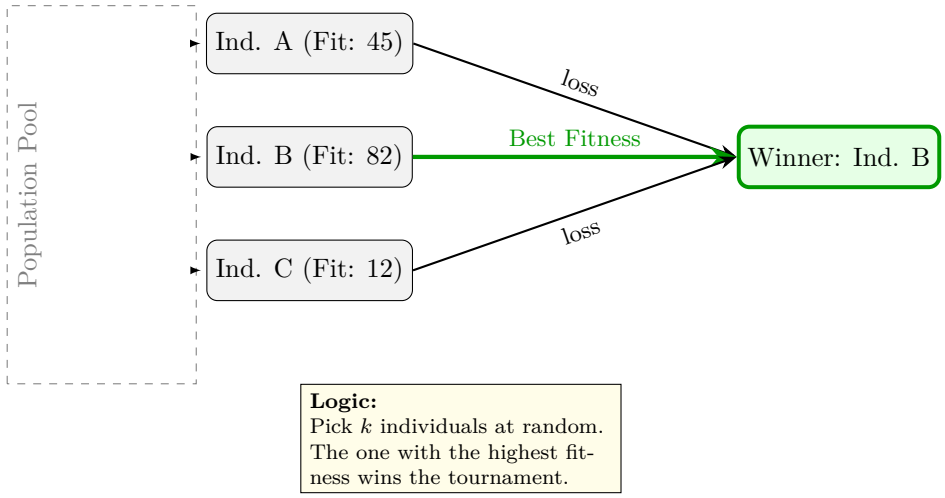


1.8.3 Tournament selection

We follow the following schema (To draw).
This can be considered as both random and deterministic.

1. It is possible (in some cases) to not evaluate all the possible individual's fitness.
2. We can change the selection pressure by changing the tournament size

Tournament Selection ($k = 3$)



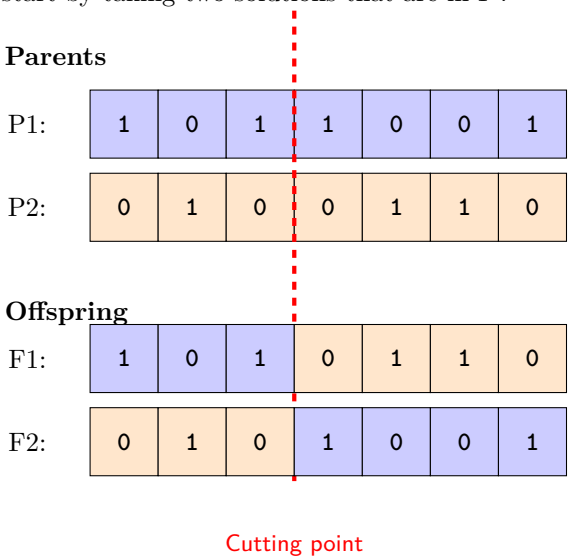
Logic:
Pick k individuals at random. The one with the highest fitness wins the tournament.

1.9 Crossover and mutation

Now we talk about the part about reproduction with variation. Selection depends only on phenotype while crossover and mutation only on genotype (DNA structure).

1.9.1 Crossover

We start by taking two solutions that are in P'.



This example is for 1-point crossover, but we can have multiple cutting points.

1.9.2 Mutation

For each character we change it with a given probability of mutation (another hyperparameter)

1.9.3 General function of genetic algorithms

Here we have a pseudocode on the general idea of genetic algorithms

1. Initialize a population P of N individuals (typically at random)
2. Repeat until termination condition is reached:
   (a) Calculate the fitness of all the individuals in the population P (can be avoided in some cases, like tournament selection). This step is computationally costly

(b) We create an empty population P'

(c) Repeat P' has N individuals

i. We choose a genetic operator (crossover with probability P_c or replication with probability $1-P_c$)

ii. Select 2 individuals from population P

iii. Apply the operator to the individuals

iv. Apply mutation to the individuals resulting from the previous step

v. Insert the individuals resulting from the previous point into P'

(d) Replace P with P'

3. Return the best individual in P in terms of fitness

Some points

- If N is odd:
  - Select 1 individual mutated and insert it into P'
  - One crossover event copies into P' only one offspring (typically the best in fitness)
- Elitism is the unchanged copy of the best K individuals in P into P' (bypassing the selection-crossover-mutation pipeline), typically K=1
- We have to do something like P →selection →P' →crossover + mutation →P" →P := P"
- Replication is the unchanged copy of the parents
- Termination condition is either
  - A good enough solution is found, and we return it
  - A maximum number of generation was performed

1.9.4 Hyperparameters of genetic algorithms

- String length →Given by the problem definition
- Population size (N) →The bigger, the better. We have an upper bound (true for every problem)
- Selection algorithm that we want to use →No free lunch holds, but tournament selection can be interesting to use (fast, and we can use the selection pressure). We can start with this and try the others. The tournament size should not be too big (2 can be good, and the minimum).
- Crossover probability (P_c) →Should be close to 1
- Mutation probability (P_m) →Should be close to 0
- Maximum number of generations →The bigger, the better but with an upper bound like for N
- Elitism on or off (and if on size of the elite and k) →Elitism should be used with K=1

1.10 Why genetic algorithms work

To answer we have to have another question →Will genetic algorithms lead to the discovery of a global optimum or an approximation of it?

To answer we have to have another question →How do genetic algorithms work?

To answer we have to have another question →What is the information that is maintained and processed in the population?

The hypothesis that we want to verify is that there are some syntactic features that give a boost to fitness. We would like to maintain these features through generations. To do this we would like to formalize a mechanism to identify structural similarities between individuals. We can call it **Schema** (set of individuals that have in common the fact of having the same characters in some positions).

A schema can be represented using strings composed by all admissible characters + 1 special character (don't care symbol, here is *). For example in the binary case, a schema is a string built using characters coming from the set {0, 1, *}.

Example
{*0000} represents both {00000} and {10000}.

Hypothesis →implicitly genetic algorithms work on schemata.

- Order of a schema →The number of characters different from *, o({011*1**}) = 4
- Length of a schema →The difference between the rightmost and leftmost positions that do not have an *, δ(011*1**) = 5 - 1 = 4

Now we want to know what schemata will survive in the population over the evolution of a genetic algorithm.

At generation t, m(H, t) is the number of individuals in the population that belong to the schema H. Given this we want to know what is m(H, t + 1). To solve this we need to know how selection, crossover and mutation change m(H, t).

1.10.1 Effect of selection on schemata

We will study the case of fitness proportionate selection.
$m(H, t+1) = m(H, t) n \frac{f(H)}{\sum_j f_j}$
Where f(H) is the average fitness of the individuals in the population that belong to schema H. In the denominator we have the average probability of selecting an individual belonging to schema H. We apply selection n times.

$$\text{AVG. FIT. OF INDIVS IN POP} = \tilde{f} = \frac{\sum_j f_j}{n}$$
$$\frac{1}{\tilde{f}} = \frac{n}{\sum_j f_j}$$
$$m(H, t+1) = m(H, t)\frac{f(H)}{\tilde{f}}$$

Schemata with an average fitness that is better than the average population fitness will increment the number of representatives and the others will decrement. At what speed?

$$\text{Assume } f(H) = \tilde{f} + c\tilde{f}$$
$$m(H, t+1) = m(H, t)\frac{\tilde{f} + c\tilde{f}}{\tilde{f}} = m(H, t)\frac{\tilde{f}(1+c)}{\tilde{f}}$$
$$= m(H, t)(1+c)$$

$$m(H, 1) = m(H, 0)(1+c)$$
$$m(H, 2) = m(H, 1)(1+c) = m(H, 0)(1+c)^2$$
$$m(H, 3) = m(H, 2)(1+c) = m(H, 0)(1+c)^3$$
$$\vdots$$
$$m(H, t) = m(H, 0)(1+c)^t$$

The growth of the number of representatives of schema H in the population is exponential with t.

1.10.2 Effect of crossover on schemata

EXAMPLE

$$A = 0\ 1\ 1 \mid 1\ 0\ 0\ 0$$
$$H_1 = *\ 1\ *\ \mid\ *\ *\ *\ 0$$
$$H_2 = *\ *\ *\ \mid 1\ 0\ *\ *$$

FOR ANY SCHEMA $P_s(H) = 1 - \frac{\delta(H)}{L-1}$

$$P_d(H_1) = \frac{5}{6} = \frac{\delta(H_1)}{L-1}$$
$$P_s(H_1) = 1 - \frac{5}{6} = \frac{1}{6} \implies 1 - \frac{\delta(H_1)}{L-1}$$

IN GENERAL

$$P_s(H) \geq 1 - p_c \frac{\delta(H)}{L-1}$$

1.10.3 Combined effect of selection and crossover

$$m(H, t+1) \geq m(H, t) \frac{f(H)}{\tilde{f}} \left[1 - p_c \frac{\delta(H)}{L-1}\right]$$

Schemata with:
– The average fitness is better the population average
– The small length will increment the number of representatives exponentially

1.10.4 Effect of mutation

The probability that 1 bit will not be changed = 1 - p_m

The probability of a schema surviving mutation =

$$\underbrace{(1-p_m)(1-p_m)(1-p_m)\dots(1-p_m)}_{o(H) \text{ times}}$$

$$P_s(H) = (1-p_m)^{o(H)} \qquad \begin{array}{l}\text{if } p_m \ll 1, \text{ then} \\ P_s(H) \approx 1 - o(H)p_m\end{array}$$

Schema theorem

$$m(H, t+1) \geq m(H, t) \frac{f(H)}{\tilde{f}} \left[1 - p_c \frac{\delta(H)}{L-1}\right] [1 - o(H)p_m]$$

We have schemata that:

- Have an average fitness better than population average
- Are short (small $\delta(H)$)
- Have a small order (small $o(H)$)

will increase the number of representatives exponentially

1.10.5 Building blocks' hypothesis

This hypothesis tells us that maintaining the building blocks in the population and making them increment exponentially will

- Maximize our chances of finding a global optimum
- Make the probability of finding a global optimum tend to 1 with time

1.10.6 K-armed bandit problem

This looks like something random but apparently is related to genetic algorithms.

Vanneschi just trashed Samuel in real time.

- Given N slot machines
- Every slot machine j has a number of arms K_j
- We assume that we already played t times and for each of this t games we know
  - The slot machine in which we played
  - The result of the game (win or lose)
- We want to find to which slot machine we should allocate our next attempts to maximize our probability of winning

Given the slot machine j, we call S_j the probability of winning using the slot machine j estimated using all the past t attempts. So $S_j = \frac{\text{Number of games where we used slot machine j and we won}}{\text{Total number of game}}$

Theorem time →If in the next game we allocate a number of attempts to slot j such that

- $S_j >$ average ($S_j > \frac{\sum_i S_i}{N}$)
- j has a small number of arms

Then

- We maximize our probability of winning
- The probability of winning tends to 1 with the number of attempts tending to infinity

The consequence is the theorem of asymptotic convergence of genetic algorithms.
Let i(t) be the best individual in the population at generation t and S_o be the best set of global optima. Then

$$\lim_{t\to\infty} P(i(t) \in S_o) = 1$$

1.11 Problems with genetic algorithms

The main drawbacks of standard genetic algorithms are

1. Premature convergence →also known as loss of diversity. For any possible problem our algorithm will produce individuals that are similar to each other. If they are not similar to the global optimum we might never reach it.
2. Position problem →typical of standard crossover. We risk splitting our good chromosomes if they are at different ends.
3. Unicity of the fitness

We can solve them with the following methods

1.11.1 Premature convergence

We have ways to measure the diversity of our population and to see if we have this problem

- Entropy →$\sum_{j=1}^{N} F_j \log(F_j)$
- Variance →$\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$

Both can be measured on genotypic (within the individual) and phenotypic (fitness) dimensions.

- Phenotypic entropy →N is the number of different fitness values in the population and F_j fraction of individuals with a specific fitness value in the population. For continuous fitness we usually define bins.

- Genotypic entropy →N is the number of different genotypes while (strings) and F_j the fraction of individuals with that genotype.
- Phenotypic variance →n is the number of individuals in the population, x_i is the fitness of individual i and $\bar{x}$ is the average fitness of the individuals in P.
- Genotypic variance →n is the number of individuals in the population, x_i (given a distance measure) is the distance of the individual i from the origin and $\bar{x}$ is the average distance to origin of the individuals in the population.

As origin, we can choose any individual (usually a string of zeroes). These methods are just monitoring systems (not a way to solve the problems).

How to maintain diversity?

We can do

1. Fitness sharing
2. Restricted mating

With fitness sharing we will give more fitness to individuals that are very diverse. Then similar individuals will be penalized. This approach doesn't change the representation of our individuals.

Restricted mating acts in the opposite way.

Fitness Sharing

For all individuals i in the population:

- Given a predefined distance measure calculate the distance from i to all the other individuals in the population
- Normalize all these distances in the interval [0, 1]
- Invert these distances (for instance calculating $S(i,j) = 1 - d(i,j)$)
- Calculate the sharing coefficient the sharing coefficient of i →$(S(i) = \sum_j S(i,j))$
- Redefine the fitness as $f_s(i) = \frac{f(i)}{S(i)}$

We can do this given:

1. A distance measure
2. A normalization method
3. A way to invert the distances

EXAMPLE

POPULATION

i_1 : 11111 $f(i_1) = 50$
i_2 : 01111 $f(i_2) = 40$
i_3 : 10111 $f(i_3) = 30$
i_4 : 00000 $f(i_4) = 10$

DISTANCE: Hamming Distance
NORMALIZATION: $d_N = \frac{d}{L}$
INVERSION: $1 - d_N$

MAXIMIZATION

New fitness i_4:

$$d(i_4, i_1) = 5 \longrightarrow d_N = 1 \longrightarrow S = 0$$
$$d(i_4, i_2) = 4 \longrightarrow d_N = \frac{4}{5} \longrightarrow S = \frac{1}{5}$$
$$d(i_4, i_3) = 4 \longrightarrow d_N = \frac{4}{5} \longrightarrow S = \frac{1}{5}$$

$$S(i_4) = 0 + \frac{1}{5} + \frac{1}{5} = \frac{2}{5}$$

The result is that we have implicit speciation.

Restricted mating

Starting from an arbitrary schema we divide it in two parts, template and functional. In the template we are basically coding what the individual will like from the mating point of view. We get that we can do cross over in a restricted way.

The model are

- Bi-directional match
- Uni-directional match
- Best partial match

RESTRICTED MATING

	⟨**TEMPLATE**⟩	⟨**FUNCTIONAL**⟩
i_1 :	$*\ 1\ 0\ *$	: 1 0 1 0
i_2 :	$*\ 0\ 1\ *$	: 1 1 0 1
i_3 :	$0\ *\ *\ 0$	: 0 0 0 0

1.11.2 Position problem

Crossover tends to destroy good individuals if the good genomes are on different sides after the cut. If we want to save those genomes we would like to reorder the indexes, but we will have problems in doing it. Our objective is to take two parents with indexes in different orders and create children that have all indexes (once). We can do something called **Cycle Crossover**

CYCLE CROSSOVER

PARENTS

	1	2	3	4	5	6	7	8	9	10
P_1 :	1	1	0	0	0	1	1	0	0	1
P_2 :	0	0	0	1	1	1	1	1	0	0

$\xrightarrow{\text{INVERSION}}$

Indices:	9	8	2	1	7	4	5	10	6	3
Values P_1:	0	0	1	1	1	0	0	1	1	0
Values P_2:	0	1	0	0	1	1	1	0	1	0

$\xleftarrow{\text{REORDER}}$

OFFSPRING

1 0 0 0 1 1 1 1 0 0
0 1 0 1 0 1 1 0 0 1

- **Step 1: Identifying a Cycle** Start with the first gene of Parent 1 (P_1) and look at the gene in the same position in Parent 2 (P_2). You then find where that value from P_2 is located in P_1[cite: 1]. Repeat this process until you return to the starting value, which completes one "cycle"[cite: 1].
- **Step 2: Mapping (Inversion)** In the example, the "Inversion" section shows the indices (1–10) rearranged to group them according to the cycles found[cite: 1]. For example, if indices 1, 4, and 5 form a cycle, they are analyzed together in this step[cite: 1].
- **Step 3: Selection and Swapping**

- For the first cycle, the offspring inherit the genes directly from their respective parents in those positions[cite: 1].
- For the next cycle, the genes are "swapped": Offspring 1 gets genes from Parent 2, and Offspring 2 gets genes from Parent 1[cite: 1].

- **Step 4: Reconstruction (Reorder)** Finally, the genes are moved back into their original numerical positions (1 through 10)[cite: 1]. The values highlighted in **red** in the image represent the specific genes that were either swapped or kept as part of a cycle to create the final *Offspring*[cite: 1].

Another way is to use **Partially matched crossover**

PARTIALLY MATCHED (MAPPED) CROSSOVER

PARENT 1

9	8	4	**5**	**6**	**7**	1	3	2	10
1	0	1	**1**	**0**	**1**	0	0	0	1

PARENT 2

8	7	1	**2**	**3**	**10**	9	5	4	6
0	0	1	**0**	**1**	**1**	1	1	0	0

OFFSPRING (Result)

9	8	4	**2**	**3**	**10**	1	6	5	7
1	0	1	**0**	**1**	**1**	0	0	1	1
8	10	1	**5**	**6**	**7**	9	2	4	3
0	1	1	**1**	**0**	**1**	1	0	0	1

Sometimes we have to use the repair algorithm if some problems are coming.
We can have a new kind of mutation where we swap the character's order inside the window.

Initial:	9	8	**4**	5	6	7	**1**	3	2	10	
							⇓				
mut1 (swap):	9	8	**1**	5	6	7	**4**	3	2	10	(swap)
mut2:	9	8	**1**	**7**	**6**	**5**	**4**	3	2	10	

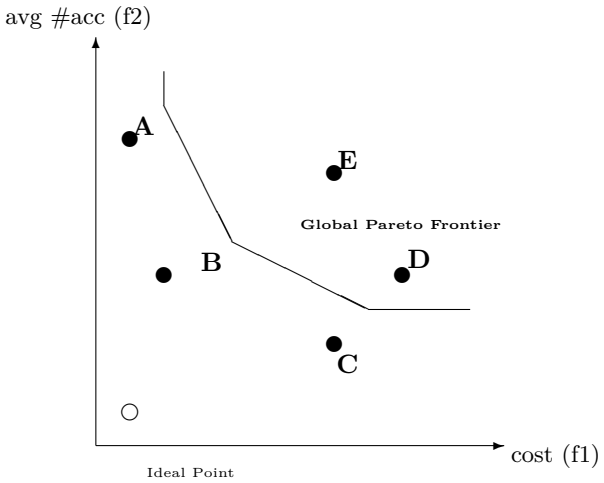
1.12 Multi-objective optimization

We might have more than one fitness to optimize, then we need a way to optimize for all of them.

MULTI-OBJECTIVE OPTIMIZATION (MULTI-FITNESS)

Fitness = (cost, avg. Numb. Of accidents)

Numeric Cases:	**Global Pareto Optimal Set** $= \{A, B, C\}$
$A = (2, 10)$	(Set of the optima)
$B = (4, 6)$	
$C = (8, 4)$	*A, B, C are NON-DOMINATED:*
$D = (9, 5)$	There is no other global solution in the
$E = (7, 8)$	search space better in all criteria.



If our solutions are not the best in all the criteria they are called non dominated. The set of non dominated solutions is called the Pareto optimal set. If all of our possible solutions are non dominated then all of them can be used, then we want to find the Pareto optimal set. But how can we choose the solution in our set?

We can plot a theoretical point to look for our preferred solution (Global optimum that doesn't exist). Since it doesn't exist then we look for the point that minimizes the distance from this preferred point. Then now we want to find an algorithm that find the closest point.

In genetic algorithms we can have multi optimization, changing selection as follows:

1. Copy all the individuals of the population into a set S
2. Find all non dominated individuals that are in S, assign a flag (1)
3. Remove these individuals from S
4. Find all non dominated individuals in S, assign a flag (2)
5. Remove them from the population and so on until S is empty
6. The flags now are proportional to the value of dominance/importance of the individuals
7. For each individual in the population the probability of selecting that individual is inversely proportional to its flag.

To implement a way to find the non dominated individuals we just look for the dominated ones. We can also have Pareto based optimization

1.13 Continuous optimization

In this setting in every position of our string we have continuous values and not integers. In this setting individuals are m-dimensional vectors of real numbers where each element is part of a given interval (optional). We can have many crossovers and mutations for this kind of algorithm. The main one are called geometric crossover and geometric mutation.

An individual can be considered as a point in an m-dimensional Cartesian space.

1.13.1 Geometric crossover

Given two parents $[x_1, x_2, ..., x_m]$ and $[y_1, y_2, ..., y_m]$ they will generate one child $[R_1x_1 + (1 - R_1)y_1, R_2x_2 + (1 - R_2y_2, ..., R_mx_m + (1 - R_m)y_m)]$ where $\forall i = 1, 2, ...m$ R_i is a random number $\in$ [0,1].

We call it geometric because if we plot the two parents we will notice that the kind lies in the middle of them (is a linear combination). The kid will not be better than the parents but for sure better than the worse one. But it can be better than both in certain situations.

1.13.2 Geometric mutation

Also known as box, or ball, mutation.

Given one individual $[x_1, x_2, ..., n_m]$ this mutation creates one individual $[x_1 + R_1, x_2 + R_2, ..., X_m + R_m]$ where $\forall i = 1, 2, ..., m$ R_i is a random number $\in [-m_s, m_s]$ where m_s is a parameter called mutation step. Geometrically we can get any element inside a given square or circle of a certain radius centered on our original point.

In the case that fitness is proportionate to a distance to a certain point (the global optimum) it is always possible to improve by creating a child that is closer to the global optimum while being inside the box. In this case we say that the fitness landscape is unimodal.

1.14 Particle Swarm Optimization

It's still a population based algorithm but is not inspired by the theory of evolution. It mimics as flocks of birds moves. It belongs to the field of swarm intelligence (when animals work together they are able to solve problems that they aren't able to if they are alone).

This algorithm is based on two laws of attraction on the single animal level

- The single animal has memory, it wants to go back to a point where it was comfortable
- There is a leader that will be followed by the rest of the animals

The individuals (particles) are m-dimensional vectors of real numbers while the population (swarm) is an n-dimensional vector of particles (a matrix).

We use the following notation

- $\vec{x_i}$ is the i^{th} individual (or position) in the swarm (i = 1, 2, ..., n)
- $\vec{v_i}$ is the velocity of particle i (m-dimensional vector of real numbers)
- $\vec{b_i}$ is the local best position of particle i (i $\in \mathbb{R}^m$)
- $\vec{g}$ is the global best position of any particle in the swarm (best so far)

The algorithm follows the following pipeline

1. For all i = 1, 2, ..., n: we initialize $\vec{x_i}$and$\vec{v_i}$
2. For all i = 1, 2, ..., n: we initialize $\vec{b_i} = \vec{x_i}$
3. $\vec{g} = \text{argmin}_{\vec{x_i}} f(\vec{x_i})$ (for minimization problems)
4. While not termination condition, for all i = 1, 2, ..., n
   (a) $\vec{x_i} = \vec{x_i} + \vec{v_i}$
   (b) Update $\vec{v_i}$
   (c) If $f(\vec{x_i})$ better than $f(\vec{b_i})$ then $\vec{b_i} = \vec{x_i}$
   (d) If $f(\vec{x_i})$ better than $f(\vec{g_i})$ then $\vec{g_i} = \vec{x_i}$
5. Return $\vec{g}$

The update of the velocity is done using the following formula:

$$\vec{v_i} = w\vec{v_i} + C_1\vec{R_1}(\vec{b_i} - \vec{x_i}) + C_2\vec{R_2}(\vec{g} - \vec{x_i})$$

Where

- w is the inertia constant, how a movement is similar to a previous one
- C_1 is the cognitive component
- C_2 is the social component
- $\vec{R_1}$ and $\vec{R_2}$ are vectors of random numbers $\in$ [0, 1]

2 Big data analytics

2.1 Introduction

2.1.1 What is Big data

Big data is a buzzword used to characterize certain kind of data. Those data were originally characterized by their volume, velocity and variety, but now we have more dimensions. Big data processing life cycle follows this path

1. Acquisition
2. Preprocessing
3. Storage and management
4. Privacy and security
5. Analyzing
6. Visualization

Usually these datasets are very huge and traditional tools can't process them efficiently. This part involves not just size but also variety, speed and complexity.

Traditional data

- Relational
- Structured
- Manageable in SQL systems

Big data

- Distributed across systems
- Structured, semi-structured or unstructured
- Requires new technology

With traditional data the best way to manage them is by vertically scaling the requirements, with big data we have to grow horizontally. Basically from using big CPUs to many small ones.

2.1.2 Data sources

The majority of big data come from the web revolution with drivers that are everyday more common

- Internet
- Smartphones
- Social media
- Sensors

Big data matters because we can discover patterns and insights and so supporting decision-making and training for machine learning models.

The data can be considered by their origins

- Human generated
- Machine generated
- Business and institutional
- Open/public

2.1.3 The Vs of Big data

The 3 Vs

1. Volume
2. Velocity
3. Variety

5 additional Vs

1. Veracity
2. Value
3. Variability
4. Validity
5. Visualization

The most common 5 Vs are Volume, Velocity, Veracity, Value and Variety.

2.1.4 The Big data ecosystem

The place where we process these data, composed of

- Storage systems
- Process engines
- Management and governance
- Analytics and visualization tools
- People and skills

The ecosystem is characterized by 4 layers

1. Storage layer
2. Processing layer
3. Management layer
4. Analytics layer

We use many tools to visualise these data (Tableau, Power BI, QlikView or Python based ones) and 3 main kinds of people act in the ecosystem

1. Data engineers
2. Data scientists
3. Business analysts

2.1.5 Applications of Big data

Big data analytics as many possible fields of application

1. Business and retail
2. Social media communication
3. Healthcare and life sciences
4. Finance and economy
5. Government and public sector
6. Smart cities
7. Science and research

2.2 Distributed storage, ecosystems and programming paradigms

2.2.1 Distributed storage and file systems

To save our data we could think about saving flat files on local disks, but this comes with problems of redundancy, slow search and retrieval and poor security. Then we can use file systems that define how files are named, stored and retrieved. Those systems use directories and store the data in blocks of given size.

As said before we need distribution to save our data in a good way, then we use many servers. To avoid problems given by common node failures we can use replication and sequential writing. The concept of connecting many independent computers to work as one big system is called **cluster computing**. Each computer runs its own operating system but when combined they form a unified resource. **Sharding** is another way where we split our big datasets into smaller pieces called shards which are then distributed across multiple machines.

Splitting our data let us ensure availability and reliability, usually we have a replication factor of 3. One of the main systems is HDFS (Hadoop Distributed File System). In HDFS we have name nodes, data nodes and replication pipelines among the data nodes.

HDFS strengths

- Efficient for very large files
- Streaming data access
- Good on low-cost pieces of hardware

HDFS cons

- Low latency access
- Lots of small files
- Multiple writers/edits

Another way to store data is using object storage (Cloud). The data are stored as objects in a bucket. Good for elastic scaling, but we have higher latency and lower throughput. Some examples are AWS and Azure Blob. This is a cost-effective way to handle many files. Replication is used as always. The advantages of this approach are

- Can deal with many files
- Small files are used
- We can scale elastically up and down
- Cheap

We also have cons

- Atomic replacement of full objects
- Lower throughput
- High latency
- Lack of query engine
- Off premises

2.2.2 Distributed computing models

Divide and conquer approach where we process data where they are stored. We scale horizontally across nodes.

2.2.3 The big data ecosystem

Formed by a web of interconnected tools for storage, processing, analytics and visualization. They will lower costs and time of deployment.

Hadoop ecosystem

- HDFS for storage
- YARN to manage resources
- MapReduce for batch processing
- Hive, Pig, Sqoop, Flume for analytics

Apache Spark

- Unifier engine for multiple tools
- Faster than MapReduce
- Multiple programming languages

The databases are usually NoSQL and can be

- Key-Value
- Document
- Columnar
- Graph

For unstructured and structured data we use data lakes, warehouses of lakehoues

2.2.4 Programming paradigms

As said we need to parallelize and resist faulting when working with big data. We can have imperative (step by step), functional (we define our own functions) and declarative (what to compute, not how) programming.

The most common approach is functional programming where computation is considered as evaluation of functions. We use things like map, reduce, filter and fold.

MapReduce paradigm

- Apply function to data blocks (Map step)
- Group by Key (Shuffle step)
- Aggregate results (Reduce step)

2.3 Distributed parallel processing and the map reduce paradigm

2.3.1 Distributed parallel processing

Using big data we deal with terabytes of data, so we can't put them on a single server. The execution will be too slow, then we will split the workload into smaller tasks that will run simultaneously and possibly in parallel.

- **Parallel** →multiple tasks executed at the same time
- **Concurrent** →multiple tasks that progress together but not necessarily at the same time

The traditional way of analyzing data is to move all the data to one machine and process them. This is inefficient, to save network bandwidth we can do a distributed approach where we send the computations where the data already is. The architecture is cluster like and made by nodes organized in racks.

2.3.2 The MapReduce paradigm

Paradigm where we try to reduce complexity. We can use Hadoop to resolve logistic problems.

Work flow

1. **Map** →Process input split and emit key-value pairs
2. **Shuffle/Sort** →group intermediate pairs by key
3. **Reduce** →Aggregate values for each key to get a final output

```
def map(key, value):
    for word in value.split():
    emit(word, 1)

def reduce(key, values):
    emit(key, sum(values))
```

2.3.3 Case study

- Temperature records
- Retail analytics

Strengths of MapReduce

- Scales to thousands of nodes
- Fault-tolerant
- Simplifies distributed programming

Limitations

- Batch only, causes latency
- Heavy disk
- Iterative and ML inefficient

2.4 The Hadoop ecosystem

2.4.1 Hadoop

Hadoop is an open source framework for distributed storage and processing of large datasets. It's designed to scale horizontally across clusters.

Key features are fault tolerance (data replication), high throughput (parallel processing) and scalability (from few nodes to thousands). Hadoop has two main pillars

1. HDFS →Hadoop distributed file system (stores data across multiple machines and replicates blocks for reliability)
2. MapReduce →processing engine (splits computation between map and reduce, extended with YARN to manage cluster resources)

Before Hadoop, we had to fit our data into one machine (limited scalability). Hadoop allowed big data storage and parallel computation.

2.4.2 Filesystems and HDFS basics

Since large datasets can't fit on one machine we have to distribute it across nodes and to make computation where data resides.

Terms of Filesystems

- **Block** →fixed-sixe unit of storage
- **Split** →logical division of input for MapReduce
- **Shard/Partition** terms for splitting datasets

Filesystems can be local (one disk, not fault-tolerant) or distributed (replicates blocks across many nodes). HDFS stores large files and is optimized for throughput (not latency), write once and read many times.

HDFS components

- NameNode →Manages metadata and block locations
- Secondary NameNode →assist with checkpoints
- DataNodes →store actual blocks

To ensure fault tolerance we have block replication, one block is stored in multiple nodes or racks. Large block size reduces metadata overhead, and we have data locality for the computation.

2.4.3 Hadoop's execution model

Hadoop originally used MapReduce v1 both as computation engine and resource manager, but there were some problems like single job tracker, limited scalability and rigidity. YARN was introduced to fix this in Hadoop 2.0.

V1

- We have a JobTracker that assigns and manages tasks.
- The TaskTracker runs the map and reduce tasks and report the status to the JobTracker.
- If JobTracker is overloaded it will break.
- As the clusters grow we have a bottleneck that will easily break the system.

YARN

- Separates resource management from job execution.
- The key roles are ResourceManager, NodeManager and ApplicationMaster
- It supports multiple processing frameworks
- Allows for better scalability, reliability and flexibility

YARN Architecture Components

- **ResourceManager (RM):** The global authority of the cluster. It handles resource scheduling (via the *Scheduler*) and job submissions (via the *ApplicationManager*). It tracks availability of CPU, memory, and I/O.
- **NodeManager (NM):** Runs on each worker node. It manages local resources, launches tasks inside **Containers**, and reports health and usage back to the RM.
- **ApplicationMaster (AM):** One instance per job. It negotiates resources with the RM and manages the job lifecycle (requesting containers, handling failures) without micromanaging every task.

Containers and Locality

- **Containers:** A logical bundle of resources (CPU + Memory + I/O). Every task runs in its own isolated container to prevent conflicts.
- **Locality Awareness:** To minimize network costs, YARN prefers **Node-local** execution (data on the same node), then **Rack-local**, and finally **Off-switch** as a last resort.

Scheduling and Benefits

- **Schedulers:** Supports *FIFO* (simple order), *Capacity* (guaranteed queues), and *Fair* (equal sharing/preemption).
- **Advantages:** Enables multitenancy, scales to tens of thousands of nodes, and supports multiple engines like Spark, Tez, and Flink beyond just MapReduce.

2.4.4 Optimization and refinements

From Splits to Reducers

- **Input Splits & Blocks:** Large files are broken into HDFS blocks (default 128 MB). Each split is assigned to a map task; parallelism depends on the number of blocks.
- **Shuffle & Sort:** Mappers produce (key, value) pairs. The system groups all values with the same key across the cluster. This step is **network-intensive** and often the bottleneck of MapReduce.
- **Reducers:** Each reducer receives one or more key groups, applies the function (sum, max, etc.), and writes the final output back to HDFS.

Optimizations: Combiners & Partitioners

- **Combiners:** Optional "mini-reducers" on mapper output. They perform local aggregation before the shuffle phase to reduce network traffic (e.g., local word count sums).
- **Partitioners:** Decide which reducer processes a specific key (default: $hash(key) \pmod{numReducers}$). Used for load balancing or specific workflows.

Handling Stragglers & Joins

- **Speculative Execution:** To handle slow nodes ("stragglers"), Hadoop launches duplicate copies of slow tasks on other nodes. The first copy to finish is kept, reducing job completion time.
- **Reduce-side Joins:** Used for joining datasets on a key (e.g., $R(A, B) \bowtie S(B, C)$). Mappers emit (B, row) from both datasets, and the reducer performs the join. Flexible but expensive due to high shuffle overhead.

Hadoop Strengths & Limitations

- **Strengths:** Handles petabytes of data, fault-tolerant (automatic replication/retries), and supported by a large ecosystem (Hive, HBase, Sqoop).
- **Limitations: Batch-oriented** (not suitable for real-time), high latency due to heavy disk I/O, and inefficient for iterative jobs (ML, graph algorithms).

2.4.5 YARN and scheduling

Resource Management in YARN

- **Resource Types:** YARN manages four key resources across the cluster: CPU (cores), **Memory** (RAM), **Disk I/O** (read/write), and **Network I/O** (bandwidth).
- **Container Allocation:** Resources are bundled into **Containers**. Each container represents a specific amount of CPU and Memory, ensuring jobs get exactly what they requested.
- **Benefits:** This provides strict **isolation** between jobs and increases efficiency by sizing containers to actual application needs.

YARN Scheduling Policies

- **FIFO Scheduler:** Jobs run in the strict order of submission. While simple, it causes head-of-line blocking where small jobs must wait for large ones to finish.
- **Capacity Scheduler:** The cluster is divided into **queues** with guaranteed resources (e.g., Department A gets 50% quota). This ensures multi-tenant access, though unused capacity can be "borrowed" by other queues.
- **Fair Scheduler:** Resources are dynamically shared so that all jobs get a fair share over time. Small interactive jobs can start immediately even if a large job is already running.

Scheduling Scenarios in Practice

- **Scenario 1 (FIFO):** A massive data-cleaning job starts first; a subsequent small query is blocked until the first job completes, leading to poor responsiveness.
- **Scenario 2 (Capacity):** A department is guaranteed 50% of the cluster. Even if other departments are busy, the quota ensures they always have access to resources.
- **Scenario 3 (Fair):** When a small job arrives during a large job's execution, the scheduler reassigns some containers, so both can progress simultaneously.

2.4.6 Hive, Pig and Sqoop

Apache Hive: Data Warehouse on Hadoop

- **Overview:** A data warehouse infrastructure built on top of Hadoop. It provides **HiveQL** (a SQL-like language), making Hadoop accessible to analysts without Java/MapReduce knowledge.
- **Schema-on-Read:** Unlike traditional databases, Hive loads data into HDFS "as is" and applies the schema only when the data is read. This offers great flexibility for semi-structured data.
- **Architecture:**
  - **Metastore:** Stores table metadata and schemas.
  - **Compiler & Optimizer:** Parses HiveQL and improves the execution plan (e.g., predicate push-down).
  - **Execution Engine:** Converts the plan into MapReduce, Tez, or Spark jobs.

Hive Optimization & File Formats

- **Partitioning:** Splits tables into HDFS directories based on a column (e.g., *year=2024*). It improves performance via **partition pruning**.
- **Bucketing:** Further divides data based on a hash function to ensure balanced distribution for joins.
- **Storage Formats:**
  - **ORC / Parquet:** Columnar formats, highly compressed, best for analytical queries.
  - **Avro:** Row-based format, excellent for schema evolution and streaming.

Apache Pig: Scripting for ETL

- **Overview:** A high-level platform for analyzing large data sets using a procedural language called **Pig Latin**.
- **Use Case:** Designed for complex data pipelines and ETL (Extract, Transform, Load) workflows. It is more "step-by-step" compared to Hive's declarative SQL approach.
- **Example Flow:** `LOAD` (bring data) → `GROUP` (by key) → `FOREACH` (apply aggregation).

Apache Sqoop: SQL-to-Hadoop

- **Overview:** A tool designed for efficiently transferring bulk data between **Relational Databases** (RDBMS) and the Hadoop ecosystem (HDFS, Hive, or HBase).
- **Functionality:** It uses Map-only jobs to import/export data in parallel via JDBC.
- **Example Command:**

```
sqoop import --connect jdbc:mysql://localhost/db --table orders
```

- **Pros/Cons:** Simple and supports incremental loads, but it is better suited for bulk transfers rather than real-time streaming.

2.4.7 Summary

- **Execution model** →How Hadoop runs MapReduce jobs
- **Refinements** →optimizations via combiners, partitioners, speculative executions
- **Ecosystem** →Hive, Pig and Sqoop

Hadoop it's still foundational but has been partially replaced by Spark for faster analytics

2.5 Introduction to Spark

Spark is a unified engine for big data processing. Some advantages of Spark are:

1. We can process in parallel large datasets across a cluster
2. We can perform ad hoc interactive queries and visualize datasets. We can also do machine learning models using MLlib
3. We can implement end-to-end data pipelines from different streams
4. We can analyze graph data sets and social networks

The success of Spark comes from the fact that is open source, fast and provides parallel and distributed computing. The interface for Python is called PySpark.
Some improvements of Spark with respect to Hadoop

- MapReduce on Hadoop is slow
- Spark is in-memory so faster and more flexible
- Spark is a unified engine for batch, SQL etc.
- Spark decouples computation from storage. Spark runs independently of HDFS and keeps data in memory between stages, avoiding repeated disk reads/writes
- Spark enables interactive queries and iterative algorithms that are difficult in MapReduce

Spark in the big data landscape complements Hadoop, works with different kind of datasets, it has cloud-native integrations, bridges between different data platforms and evolved after Hadoop improving it.

Spark supports multiple languages like

- Scala
- Python
- Java
- R
- More via 3rd party APIs

All languages run on the same Spark core engine ensuring consistency and scalability. The choice on what to use depends on skills, integration needs and project goals.

Spark provides two levels of APIs for working with data

- Low-level (RDDs) →fine-grained control, defines transformations and actions
- High-level (DataFrame and Dataset) →structured, optimized queries using Catalyst optimizer and Tungsten engine

RDDs are ideal for unstructured or experimental workloads while DF and DS are better for performance and interoperability. Users can mix both depending on their data workflow.
Some useful Spark applications are Batch ETL, interactive SQL queries, streaming analytics, ML at scale and graph processing.

The main Spark architecture is made of the followings

- Driver program →orchestrates the operations (runs SparkContext, defined RDD transformations and actions and collects results)
- Cluster manager →YARN, Kubernetes and Mesos
- Executors →run tasks and store data (worker processes on cluster nodes, executes tasks in parallel and cache data in memory for reuse)

SparkContext is an old entry point for Spark application, it creates RDDs and manages jobs. It was introduced in early Spark, and it is the core connection to the cluster. This by communicating with the cluster manager to allocate resources. The RDD API depends on the SparkContext for all operations.

SparkSession is a higher-level entry point that unifies SQL and RDDs APIs.

In Spark, we have two main operations

1. Transformations →define a new RDD based on the current one(s). The data inside are never changed, but we want to modify them as needed. These transformations are lazy (they won't be executed until action). Some common transformations are map, filter and flatMap.
2. Actions →return values to the driver or save to storage. Some actions are count, take, collect and SaveAsTextFile

The evaluation is lazy since transformations build a plan while actions trigger the execution.

RDD →Resilient distributed dataset, it's an immutable, distributed collection of objects built via transformations. RDD properties are immutability, lazy evaluation, fault-tolerance and partitions across cluster. We can create RDDs from collections and data in memory, from external storage and from transforming existing RDDs.

Spark has two main level of execution

1. Logical →what operations to perform, built when RDD operations are defined, represented as a DAG of transformations
2. Physical →how Spark executes the plan, it splits the DAG into stages and tasks, the DAG scheduler and Task scheduler optimize and distribute the work

Lazy evaluation follows this procedure

- Spark builds DAG of operations
- Optimizes before execution
- Saves redundant computation

Creations and transformations do nothing on their own. They are applied when an action is launched.

Transformations on RDD can be by Type

- Unary →operate on one RDD and obtain one new RDD, usually for element-wise operations like filter, map and flatMap
- Binary →combine two RDDs, like union, intersection and subtraction
- Pair →work on (key, value) pairs, like groupbykey, reducebykey and sortbykey

We can also define narrow and wide transformations based of the execution implications

- Narrow →each output partition depends on one input partition
- Wide →the output depends on multiple input partitions, they trigger shuffles (expensive operations)

For ML algorithms some important elements are persist() and cache() because they store RDD in memory.

To ensure fault tolerance RDDs follow a lineage graph, so if the partition is lost we can recompute from lineage. It's a built-in recovery mechanism that uses a DAG that records how an RDD was created. We don't store data but we recompute them.

Different from Spark (in-memory, DAG scheduler and faster) MapReduce is disk-based, simpler and robust.

The current system supports several cluster managers like Standalone mode, YARN and Kubernetes.

RDDs are limited by some elements

- Verbose API
- No schema
- Less optimized than DataFrames

2.6 DataFrames in Spark and Spark-Hadoop relationship

2.6.1 Recap

In the previous section we saw the concept of RDDs that are distributed and fault-tolerant collections. But they are also limited by the fact that they have no schema (so they are inefficient for queries) and that the API is verbose. Our next step will be to review DataFrames since they are structured and optimized (SQL-like).

2.6.2 DataFrames

A dataframe is a distributed table-like structure with rows and named columns. Pandas DF and SQL tables inspired them. Each column has a defined type. These DataFrames are built on top of RDDs, but they are faster (in-memory) and easier (table-like).

With an RDD we tell Spark how to aggregate, while with a dataframe we communicate what we want and Spark will optimize it.

On top of that DataFrames have the following benefits

- Easy to use
- Performance are good
- Interoperability
- Integration

To create a DataFrame we can start from

- Structured files
- RDDs with schema
- External database

The schema can be inferred by Spark or defined by the user.

We recall transformations (return a new dataframe) vs actions (trigger execution). They follow the schema Transformations →DAG →Actions.

Some very useful functions are called Window functions →They enable analytics like top-N per group.

The queries are optimized by Catalyst in the following stages

1. Analysis
2. Logical optimization
3. Physical planning

4. Code generation

After a query is written Spark builds a logical plan that describes what operations need to be done. We then get the unresolved logical plan that includes references Spark has not yet verified. Catalyst then analyzes the plan using the schema and the metadata creating an analyzed logical plan. This last plan is independent of any physical execution strategy.

The optimization phase follows these steps

1. The Catalyst Optimizer transforms the analyzed logical plan into an optimized logical plan
2. It applies both rule-based and cost-based optimization
3. Predicate pushdown (move filters close to the data source)
4. Projection pruning (remove unnecessary columns)
5. Constant folding (simplify expressions)
6. Null elimination and common subexpression removal
7. The result is an optimized logical plan that preserves semantics but reduces computation

The physical plan defines how Spark will perform the computation

- Catalyst's planner takes the optimized logical plan and selects the most efficient execution strategy (SortMergeJoin, BroadcastHashJoin, ShuffleHashJoin, whole-stage code generation, task parallelization and stage pipelining)
- We can have multiple candidate physical plans and Spark will choose the best one
- Tungsten engine receives the plan, and it will execute it managing the memory

Tungsten executes the physical plan using whole-stage code generation and off-heap memory management

- Memory + CPU optimizations
- Off-heap memory management
- Whole-stage code generation
- Vectorized processing

Catalyst →handles query analysis and optimization ensuring that Spark finds efficient execution strategies automatically

Tungsten →handles execution and performance optimizing CPU and memory usage for physical plan execution.

These two together form the Spark SQL engine. This engine makes Spark efficient, declarative and adaptable. Spark can be integrated with SQL providing the ability of working with different types of data and formats.

In this setting the DataFrames are the API and Spark SQL is the interface.

Performance tips

- Use Parquet/ORC instead of CSV
- Partition data by common query keys
- Cache DataFrames when used
- Broadcast joins for small tables

From this we can have the takeaway that DataFrames are preferable to RDDs unless RDD operations are needed.

Use RDDs if

- You need low level transformations

- You work with unstructured or complex data types
- You require fine-grained control over partitions or persistence

Use DataFrames when

- Data is structured
- You want automatic optimization
- Concise SQL-style queries are preferable

For Python users DataFrames are better. They can resort to RDDs if they need more control.

2.6.3 Datasets

Spark has also the option of using datasets, a unified API that combines RDDs and DataFrames. Some benefits are that datasets are type-safe, support both functional and relational operations, available in Scala and Java (not Python) and we can have optimization via Catalyst and Tungsten.

2.6.4 Spark-Hadoop relationship

Spark integrates with Hadoop but doesn't depend on them. It can also run standalone or in cloud environments. The read and write come from HDFS, YARN is for resource management, Hive Metastore for table metadata and SQL queries, MapReduce is replaced for in-memory executions.

The integration is done

- On YARN
- With Hive
- With HDFS

2.7 Spark in Cloud-based computational environments

The main use of Cloud computing in the context of big data is given by the fact that is elastic and can be used with on-demand options. It's the ideal way to work with distributed data processing. It also removes hardware management overload and some models support pay-as-you-go models.

Running Spark in the cloud is also useful because pre-existent clusters can be costly, too rigid, and they require skilled admins. With cloud based nodes on the other hand with more elastic and scalable options. Basically we don't need to manage our clusters manually, we can scale easily, the integration with cloud-native storage is easier, and we can access prebuilt Spark distributions.

Some benefits of this approach are

- The deployment is easy and so the management
- We can integrate everything with cloud-native tools
- Team collaboration is optimized

Some cloud deployment options are

- Managed Spark services
- Spark on Kubernetes or standalone clusters
- Interactive notebooks

Spark can be managed in two main ways: Managed and Self-Managed

Managed Spark

- Automatically handles setup, scaling and monitoring
- Best for production and teaching

Self-Managed Spark

- Full control over configuration and cost
- Best for research and experimentation

2.7.1 Databricks

Databricks is a unified platform for data engineering, analytics and AI. It was created by John Spark (the inventor of Spark). This platform is able to fully manage Spark clusters with a notebook based environment. Databricks is formed by the following components:

- Workspace →collaborative notebook environment
- Clusters →auto-scaling Spark clusters
- Jobs →automated workflows
- DBFS →Databricks File System (built on cloud storage)

Databricks has the following advantages

- One-click cluster creation
- Integrated visualization and SQL interface
- Optimized Spark runtime
- Supports all major clouds

Databricks follows this workflow

1. Upload data to the cloud
2. Create the Databricks notebook
3. Initialize Spark session
4. Read, transform and visualise data
5. Export the results or train models

Simplifying to **Raw →Clean →Analytics**.

Databricks can be used with its free edition (not for production tho).

2.7.2 AWS Elastic MapReduce (EMR)

AWS EMR is an Amazon-managed Hadoop and Spark platform that uses EC2 instances and S3 for storage. These two provide resizable virtual services and object storage services.

EMR clusters are made of

- Master node →job coordination
- Core nodes →run HDFS and Spark executors
- Task nodes →compute-only

On EMR Spark environments are pre-configured, and they can be easily integrated with S3. EMRFS allows seamless S3 access.

The workflow is the following

1. Store data in Amazon S3
2. Launch EMR with Spark application
3. Process the data and write the results back to S3 or RedShift (data warehouse)

S3 →EMR →S3 →RedShift

EMR has the following advantages

- Flexible cluster customization
- Cost-control via auto-termination
- Deep integration with AWS analytics tools like Athena and Glue

2.7.3 Google Cloud Dataproc

Dataproc is a tool that help us manage Hadoop and Spark services on Google Cloud. It can be integrated with services like GCS (data lake), Google BigQuery (Analytics) and Vertex AI (unified ML platforms).

Dataproc is convenient since it help us for fast cluster startup, pay-per-second billing is enabled and some tools like Jupyter, Spark and Hadoop are already installed.

Dataproc uses compute engine VMs for nodes, the storage is done via GCS (not HDFS) and Job orchestration is done via Cloud Composer (Airflow).

2.7.4 Azure HDInsight

HDInsight is a Microsoft's managed big data service. It supports Hadoop, Spark, Hive, Kafka and HBase and has a tight integration with Azure Data Lake. The key features of HDInsight are its good security, the fact that can be easily scaled through Azure and that it can be integrated with Power BI and Synapse.

A typical use include the following steps

1. Data from Azure Data Lake
2. Spark job on HDInsight
3. Visualization in Power BI

HDInsight has some pros like a very good Microsoft integration (how is this a pro?) and the cluster management. On the other hand the startup is slow and Databricks is cheaper.

Some replacements that we can find in Azure are Databricks, Fabric, Azure Synapse Analytics, Azure Data Factory, HDInsight on AKS, Azure Stream Analytics and Azure Data Explorer (Kusto).

2.7.5 Kubernetes

Since Spark can run natively on K8s clusters we have some advantages if we want to use Kubernetes. Some of them are that we don't care about which cloud service we use, we have fine-graned container scheduling, and we use easily hybrids of cloud services. But we require some DevOps skills, and we have to manually tuning our resources.

2.7.6 Google Colab

Colab is a free, simple and web-based environment that is very useful for learning and experimenting. For storage, it can be connected to Google Drive. Spark can be installed easily on Colab.

Some problems with Colab

- Not suitable for large datasets
- No persistent cluster
- Good for demonstrations or teaching only
- For production we need dedicated or enterprise servers.

2.7.7 Lightning AI

Lightning AI is a cloud platform for distributed AI and data workflows. It can be integrated with Spark for large-scale preprocessing. It's focussed both on ML pipelines and not just ETL. In this integration Spark handles data ingestion and transformation while Lighting handles model training and orchestration. It's very suitable for AI labs and teaching advanced pipelines.

Load data from S3 or GCS →Use Spark to preprocess →Lightning AI trains model on cleaned data

Lightning AI supports GPU acceleration, auto-scaling and monitoring and Jupyter-like notebooks

2.8 Performance and data engineering in Spark

2.8.1 Why Spark jobs are slow

Different ways of running a query can have different results in terms of execution cost. Useless aggregations can result in worse performance, the use of filters can be advisable. In Spark moving data is more expensive than computing on it and by this Shuffle is the main bottleneck in terms of performance.

The golden rule in Spark to improve efficiency is to reduce data and movement, this can be achieved by

- Filtering early
- Select only needed columns
- Avoid unnecessary shuffle

2.8.2 How Spark optimizes queries

Spark does not optimize the queries as written instead it analyzes the query, it rewrites and chooses the most efficient execution. Every Spark job follows the pipeline

1. Query
2. Plan
3. Optimize
4. Execute

Usually Spark tries to reduce rows early, reduce columns, minimize shuffle with the goal of reducing data movement. For the Join Spark chooses Broadcast Join or Shuffle Join.

2.8.3 Data engineering for performance

Good performance is obtained by following these principles

1. Reduce data size
2. Avoid shuffle
3. Use efficient formats
4. Reuse computation
5. Use correct join strategy

One good format is Parquet, different from CSV because is columnar, compressed and fast. Data can be organized by key and by partition pruning.

Another useful element is caching, we save some results in the memory to reuse them. We should avoid caching if we are in a one-time use setting, data are very large and the memory is limited.

Joins are expensive operations, and we can have two types of them

- Broadcast →Small table sent to all nodes, large table stays distributed
- Shuffle →Large tables require redistribution, expensive because data move across nodes

If we apply repartition we have shuffling, with coalesce we don't.

Uneven data distribution can also be problematic since if one node is overloaded we can have a bottleneck. To handle this case

- Repartition
- Filtering early
- Key salting

The size of the files has an impact too, too many small files means overhead while too large files means poor parallelism.

The ETL pipeline

1. Raw
2. Clean
3. Transform
4. Store
5. Query

Real pipeline example

1. S3
2. Spark
3. Parquet
4. BI

The processing of the data can be periodic (Batch) or in real-time (streaming).

Performance checklist

- Use Parquet
- Filter early
- Select needed columns
- Avoid shuffle
- Use caching wisely

Common mistakes

- SELECT *
- Too many joins
- No partitioning
- Ignoring skew

2.9 Big data storage technologies and relation with Spark

2.9.1 Intro to big data storage

Given the scale and diversity of big data we can't use traditional relational databases. The problems come from the volume, the velocity and the variety of this kind of data. We need a scalable, distributed and flexible way to store our data. Most modern solutions employ horizontal scalability, fault tolerance, flexible schema and integration with different engines like Spark and Hadoop.

Recall that HDFS splits the data in block of 128 MB across nodes, and it's ideal for large sequential reads (batch analysis) but not for small and real-time queries. NoSQL technologies arose to handle these new workloads, we have the main 4 families:

1. Key-value Stores
2. Document Stores
3. Column-family store
4. Graph databases

Recall the CAP theorem, BASE and ACID principles from storing and retrieving.

Since no single database fits every use case modern architectures combine multiple storage types, this approach is called polyglot persistence.

2.9.2 Key-value DB

This is the simplest form of a NoSQL database. The data are stored as a pair key:value. It's optimized for constant-time lookups, it's schema-free and extremely fast. Some common systems are Redis, Amazon DynamoDB and Riak. They key must be unique, and the values can be of different types. Redis keeps the data in memory, so the operations will be very fast.

Some operations are

- SET key value →store data
- GET key →retrieve
- DEL key →delete
- INCR key →increment counters
- EXPIRE key seconds →set time-to-live

Strengths

- Extremely fast reads/writes
- Simple data access model
- Scales horizontally

Limitations

- No complex queries and joins
- Limited analytical capability

The best use cases are caching, gaming and high-traffic web apps. This approach generalizes the concept of Associative Keys to databases.

2.9.3 Document DB - MongoDB focus

This kind of databases store data as JSON-like documents. The schema is flexible, and we can easily query using fields and indexes. Some examples are MongoDB and Couchbase. It's ideal for semi-structured data such as catalogs, users and logs. MongoDB is the most popular document database. It's open-source, widely adopted and stores data and BSON (binary JSON). It balances performance, flexibility and scalability. Used by organizations like eBay, Forbes and CERN.

The hierarchy is the following

1. Database
2. Collections
3. Documents

Even though the schema is not fixed we can still validate it. Every MongoDB document needs to have a '_id'.

The usual operations are CRUD (create, read, update and delete)

1. Insert
2. Read
3. Update
4. Delete

Using indexes we can improve the query performance. Indexes can be

- Single-field
- Compound
- Text and geo-indexes

The aggregation in MongoDB works like a pipeline with some common stages like $match, $group, $sort and $project.

Replication in MongoDB ensures data redundancy and fault tolerance by copying data across multiple servers, called replica sets. A replica is made of primary node (handles all write operations) and secondary node (replicate data from the primary asynchronously). If the primary fails, one secondary becomes the new primary (automatic failover) and the clients continue reading from any node. The benefits of this approach are high availability, fault tolerance, data redundancy and read scalability.

Sharding is another tool where data are distributed across multiple servers (shards) so that no single node has to handle all the load or store all the data.

1. Data is split into chunks based on a shard key
2. Each chunk is stored on a different shard
3. A config server keeps track of where each chunk resides
4. A mongo's router direct client queries to the appropriate shard

The benefits are horizontal scalability, balance of read and write load across nodes and geographical distribution is supported.

In production, we usually combine replication ad sharding to make global scalability possible. Each shard is a replica set, this provides both scalability and fault tolerance.

MongoDB has some strengths (flexible schema, JSON integration, great for rapid prototyping and rich aggregation features) and limitations (joins are limited, ACID are weaker, and we require index management)

The main ecosystems related to MongoDB are

- MongoDB Atlas
- Compass
- BI Connector
- Spark Connector

2.9.4 Wide column DB

This typology of database store data in rows and dynamic columns grouped into column families. It's inspired by Google's Bigtable. Some examples are Apache Cassandra and Apache HBase. The first one doesn't rely on Hadoop while the second one does. Wide columns DB are best for high-write, time-series and IoT data.

The table structure is like relational databases but with more flexibility and scalability. Each row will have a different set of columns, and columns are grouped into column families rather than fixed tables (like a nested key-value store). Data is stored by column rather than by row. This structure is optimized for aggregations on specific columns. The usual Cassandra operations are Create, Insert, Query and Delete.

As always we have clusters (here peer-to-peer with no master node), data is replicated across nodes for fault tolerance, and we have tunable consistency (one, quorum and all). We can see Wide columns DBs as a collection of spreadsheets where each row have its own set of columns depending on what data it has.

Some design guidelines are

- Denormalize instead of join
- Use both column names and column values to store data
- Model an entity with a single row

2.9.5 Graph DB

Graph databases store data as nodes, edges and properties in order to traverse relationships easily. Some examples are Neo4j, JanusGraph and TigerGraph. With this structure queries can be fast and intuitive. Some easy common use cases are social networks, fraud detection, knowledge graphs and recommendation systems.

2.9.6 Other DB types

We also have additional kinds of databases like

- Search databases →Optimized for text search and analytics. Some examples are Elasticsearch and Apache Solr. Inverted indexes are used for fast keywords and full-text queries. These are great for log analytics, monitoring and search engines.
- Time-series databases →Specialized for sequential data. Some examples are InfluxDB, TimescaleDB and Prometheus. This structure is optimized for regular insertions and aggregations over time windows
- Vector databases →New generation databases for AI and similarity search. Some examples are Pinecone, Weaviate, Milvus and FAISS. The database stores embeddings of our numerical vectors and can be used for semantic search, recommendation and image/text similarity.

2.10 Machine Learning in Spark

2.10.1 Introduction

As we know machine learning is the process of extracting pattern from data using statistics, linear algebra and numerical optimization. It can be applied to many problems. We have the following types of machine learning

- Supervised
- Unsupervised
- Reinforcement learning
- Semi-supervised
- Self-supervised

Here we will see mainly supervised and unsupervised. Then we will talk about classification, regression, clustering, dimensionality reduction and association rules. Some algorithms in Spark MLlib for unsupervised learning are

- K-means
- Latent Dirichlet Allocation

- Gaussian mixture models

We want to use machine learning in Spark because our big data are too large for single machine machine learning. Spark MLlib is a scalable ML library built on top of Spark. Some key features are

- It works on distributed DataFrames
- Supports both supervised and unsupervised machine learning
- Integrates with pipelines

Originally it worked with RDDs, now is dataframe based. This library provides

- Transformers
- Estimators
- Pipelines

The advantages of MLlib are

- Distributed training on clusters
- Compatible with Spark SQL and DataFrames
- Integration with streaming and GraphX
- Used in production

2.10.2 Spark MLlib concepts

Transformers
These are functions that transform DataFrames using fixed rules. They do not learn from the data and implement the transform(). We get input→output columns. These functions are usually used in preprocessing. Some examples are Tokenizer, VectorAssembler, HashingTF, StandardScalerModel.

Estimators
These are algorithms that fit models and learn from the data. Implement the fit() method that in return gives us a model. Some examples are LogisticRegression, Kmeans and LinearRegression.

A special case is the one of estimators that learn transformations. They basically learn on how to transform data for feature engineering. They implement with fit() and produce a Transformer. Some examples are StringIndexer, OneHotEncoder and StandardScaler.

Pipelines
Pipelines are chains of transformations and estimators used to standardize the ML workflows. For example preprocessing →model training →evaluation.

Feature engineering
In Spark MLlib we use feature engineering to transform raw data into meaningful numerical features via indexing, encoding, scaling and assembling in order to improve our model performance in distributed ML pipelines. Some common transformers are

- StringIndexer →Categorical encoding
- VectorAssembler →Combine features
- StandardScaler →Normalize data

2.10.3 Algorithms

For classification

- Logistic regression
- Decision trees
- Random forests

For regression

- Linear regression
- Gradient boosted trees

For clustering

- KMeans
- Gaussian mixture

Spark MLlib includes ALS (Alternating Least Squares)

2.10.4 Evaluation

In MLlib we have the following metrics

- Accuracy
- Precision
- Recall
- RMSE
- R^2
- Silhouette score

Recall that we have to split our dataset into train, test and validation and evaluate the model with the right transformations avoiding leakage. We can apply cross-validation.

2.10.5 Advanced Topics

Spark MLlib is mainly for classical machine learning, to perform deep learning tasks we can integrate TensorFlow and PyTorch using

- BigDL →run DL models directly in Spark clusters
- Horovod →enables distributed training of TensorFlow and PyTorch models using Spark for data preprocessing and coordination.

We can apply these models on real-time data to work with streaming.

MLlib has some limitations on the side of deep learning support, the algorithms are not flexible and debugging on distributed nodes can be tricky.

2.11 Pipelines in Spark

2.11.1 Introduction

As we know the ML workflows involve multiple steps like cleaning, engineering training and evaluation. Without pipelines the code can be messy, with them, we can standardize it by using modular workflows. The key idea is that a pipeline is a sequence of stages, mainly transformers (to process data) and estimators (to learn from these data). The idea is similar to the one present is Scikit-learn but applied to big data.

2.11.2 Spark pipeline API

The transformers stage include taking a dataframe as input and to output a transformed version of it. We want to apply fixed rules to implement to transform function. We are not learning from the data.

The estimators phase on the other hand is made of algorithms that learn from data. They implement the fit function (it produces a model). We also have some special cases of estimators that learn transformations, their fitted models are the transformers used in pipelines. These estimators will learn rules to do feature transformation but not to do predictive tasks. We must fit them before the use.

As we know a pipeline is an ordered list of stages where the data flows through each one in order. Pipelines are good because they are

- Modular and reusable
- Automates repetitive steps
- Scales across distributed Spark clusters
- Encourage clean ML engineering practices

2.11.3 Feature engineering pipelines

Some useful pipelines can be

1. **StringIndexer + OneHotEncoder** →To convert categorical values to numbers or to binary encode them
2. **VectorAssembler** →Combines multiple feature columns into a single one
3. **StandardScaler** →Standardizes the feature into an N(0, 1) fashion to help our algorithms converging faster

2.11.4 Model training and evaluation

To train a pipeline we use the fit method, it will create a pipeline. To apply it we use the transform method. The evaluators provided by Spark MLlib are for classification, regression and clustering. We can evaluate a model using the usual metrics.

2.11.5 Hyperparameter tuning

To improve our models we can use some built-in functions such as grid search, cross validation and the split in validation and training.

2.12 Graph Analytics

2.12.1 Introduction

As we know many real-world problems involve networks and traditional databases are not optimized for this kind of relationships. Using graph analytics we can get insights from nodes and edges. The basic terms of graphs are the following:

- Node (or vertex) →entity
- Edge →relationship between nodes
- Directed/Undirected, Weighted/Unweighted

We can apply graph analytics to do tasks like

- Community detection
- Suspicious pattern detection
- Link prediction
- Path optimization
- Interaction network

2.12.2 Spark GraphX

One native Spark graph analytics library is GraphX. It's built on top of RDDs, and it combines both ETL and graph computation.

We consider a graph as a pair of RDDs and with this library we can use graph operators and algorithms. Some common operations are

- mapVertices/mapEdges →transform attributes
- joinVertices →enrich data
- subgraph →filter

- triplets →combine vertices + edges

In GraphX we have some built-in algorithms that can be used, like:

- PageRank
- Connected Components
- Triangle counting
- The Shortest Paths
- Label propagation (communities)

2.12.3 GraphFrames

Since we can use GraphX only with Scala and since it's mainly RDD based we would like to have a framework to work with DataFrames and Python. Luckily we have GraphFrames. It also integrates well ML and SQL.

We can explore our graph using graph queries. If we want to do some pattern matching operation we can use something called motif finding. Additional operations are for computation of degrees, aggregations, page rank (node importance), connected components (find group of nodes), label propagation (find communities) and breadth first search (the shortest path, reachability and relationship discovery).

2.12.4 Advanced topics

Graph features can be used inside ML pipelines for different tasks. These graphs can be used for streaming of data since they can evolve dynamically.
The storage options are usually:

- HDFS
- Cassandra
- Neo4j

With these structures we can encounter problems if our graphs are huge since we can fall inside problems with our partitions. Also, the algorithms might not parallelize well. Some useful ways to solve the problems are partitioning, caching, check pointing and query optimization.

2.13 Streaming

2.13.1 Introduction

Traditional batch processing is done on bins of hours or days, but many modern apps require insights done in real time. In this setting streaming analytics allow us to process events as they arrive.

Before Spark most engines used a record-at-a-time model, one by one. This approach is fast but difficult to operate at scale. Spark then introduced a new model to simplify scaling and reliability. This approach is based on micro-batch processing where data is grouped into small batches. Each batch is processed using deterministic and parallel tasks. The key advantages are that we have a strong fault tolerance, exactly once semantics and deterministic compute are ensured and operations are simplified. Latency is a bit higher but still suitable.

2.13.2 Structured streaming overview

Structured streaming is built on Spark SQL and DataFrame API. The key idea is that we consider streaming as incremental batch, so developers write queries as if static while Spark run them continuously.

Structured streaming treats a data stream as an unbounded table (a new event is a new row appended to this table). The developer will write a normal DataFrame or SQL query on that table (as for batch) while Spark incrementalizes the query, each micro-batch updates the result table. The triggers decide when to update the result and sinks decide where it is written.

2.13.3 Streaming concepts

Some useful concepts about streaming

- **Micro-Batch model** →Spark processes small batches of data, we want a balance between throughput and latency. This is transparent to developers
- **Event time vs Processing time** →Event (when the event occurs) or processing (when Spark processes it). Structured Streaming handles late data with watermarks
- **Windowing** →Data are grouped into time windows
- **Exactly-one semantics** →Spark guarantees no duplicate processing even when failures happen, this is important for finance and transactions. We have it when we have replayable source, deterministic transformations, idempotent or transactional sink and a stable checkpoint directory
- **Checkpointing and fault tolerance** →States are stored in checkpoints, this allows recovery after failure. Key for production streaming jobs

2.13.4 Spark structured streaming API

Usual input sources are Kafka, Socket, files and cloud storage. Output sinks usually are console, files, Kafka and databases.

Kafka

Kafka is a distributed publishing/subscribe platform designed for horizontal scalability. It's often used as a scalable buffer for streaming data into and out of Hadoop storage systems. Acts as middleware that guarantees durable ingestion from diverse data sources. Distributed sequential logs are used to store messages allowing clients or consumer groups to read from any position via simple numerical offsets. Output modes can be append, complete and update.

As we know structured streaming keeps a result table that is updated every trigger. The output mode controls which rows from that table get written to the sink.

Append

- Writes only new rows to the result table since last trigger
- It requires that those rows will never change later
- Typical queries are stateless transforms and event-time window aggregations with watermark
- Good for append-only sinks like files or data lakes

Update

- Writes only rows whose values changed since the last trigger
- Can output the same key multiple times as aggregates evolve
- Typical queries are stateful ones
- It's good for sinks that can handle upserts and overwrites

Complete

- It writes the entire result table every trigger
- Only practical when the resultant table is small
- Typically used for debugging, dashboards or in-memory tables where a full refresh is cheap
- Not ideal for file sinks

With this information we should use append for immutable and append only results (files), update for changing aggregates with updatable sinks and complete only when the result is small, and we want a full refresh each time.

A structured streaming recipe follows this pipeline

1. Define the source
2. Transform the DataFrame
3. Choose sink + output mode
4. Configure trigger + checkpoint
5. Start and monitor the query

2.13.5 Advanced streaming

Some ways to improve our streaming can be using triggers (control how often the streaming query processes new data), use stateless (no stored history) or stateful (keeps running state) transformations, use the right aggregations and joins, use ML models and graph analytics. For performance and scaling we can tune batch interval, use state TTLs, prefer append mode, compress checkpoints and monitor latency.

2.13.6 Challenges and best practices

Some challenges in Streaming can be

- Out-of-order events
- Late arrivals
- Fault tolerance at scale
- Cost of always-on clusters

Best practices

- Use watermarks for late data
- Keep stateless when possible
- Monitor with Spark UI and logs
- Test with small replayable streams

To monitor our streaming apps we can use metrics embedded in Spark UI, do some latency monitoring and integrate with Prometheus or Grafana

3 Neural and evolutionary learning

3.1 Machine learning review

The traditional computational method involves the following pipeline: Problem →Algorithm →Program. When we encounter a problem for which it's difficult to come up with an algorithm we can go towards machine learning based approaches. In these approaches we try to give the computer the ability to learn how to solve those problems.

A good way of learning is improving thanks to our own experience, a cycle where we do mistakes, we try to understand what was the mistake, why we did that mistake, and we try again while getting a reward whenever we do something good. Our systems will basically do this. In the context of machine learning we always try to learn a function.

The problems always involve a set of data and a function that tries to be as similar as possible the one that generated the initial set of data D.

$$\forall i \in \{1, 2, \ldots, N\}, \quad \Phi(x_{i1}, x_{i2}, \ldots, x_{iM}) = y_i$$

Where

1. Φ →is our target function
2. **D** →is our dataset
3. We call learn the process that allows us to find a function **f** that approximates Φ
4. **f** →is the function returned by the learning process and is called data model
5. $y_1, y_2, ..., y_M$ →are called target values

If the target values are known we talk about supervised learning, if they are unknown we are talking about unsupervised learning. Our function f should have good generalization ability (it behaves like Φ also with data that do not belong to D). This is a crucial concept in machine learning.
If this generalization ability is not good we say that f overfits the data, and this issue is known as overfitting. Generally we assume that our data are generated following a certain process and that we have knowledge hidden in our data. We want to learn this knowledge and build follow-up knowledge that can be used with data that are outside our training set.

The process of learning can be applied to two main tasks:

- Classification →The co-domain of the target values is a discrete set of limited dimensions called classes. We want to partition our data into these classes
- Regression →We try to find a function that receives the data and tries to output a continuous value

When we receive new data in a supervised setting we apply the function to the new data, and we accept the result (prediction). In the case of unsupervised we insert our new datum inside the class that is most similar to it.

3.1.1 Performance measures for regression

- **Mean Absolute Error (MAE)**

$$MAE = \frac{1}{N} \sum_{i=1}^{N} |y_i - \hat{y}_i|$$

  where:

  - N = total number of observations
  - y_i = actual (true) value of the i-th observation
  - $\hat{y}_i$ = predicted value of the i-th observation
  - $|y_i - \hat{y}_i|$ = absolute error between actual and predicted value

- **Mean Squared Error (MSE)**

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

where:

  - N = total number of observations
  - y_i = actual (true) value of the i-th observation
  - $\hat{y}_i$ = predicted value of the i-th observation
  - $(y_i - \hat{y}_i)^2$ = squared error between actual and predicted value

- **Root Mean Squared Error (RMSE)**

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}$$

where:

  - N = total number of observations
  - y_i = actual (true) value of the i-th observation
  - $\hat{y}_i$ = predicted value of the i-th observation
  - $(y_i - \hat{y}_i)^2$ = squared error
  - $\sqrt{\cdot}$ = square root operation

- **Pearson Correlation Coefficient**

$$r = \frac{\sum_{i=1}^{N}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{N}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{N}(y_i - \bar{y})^2}}$$

where:

  - N = total number of paired observations
  - x_i = value of the first variable for the i-th observation
  - y_i = value of the second variable for the i-th observation
  - $\bar{x}$ = mean value of variable x
  - $\bar{y}$ = mean value of variable y
  - r = Pearson correlation coefficient

MAE is not very sensitive to outliers in comparison to MSE since it doesn't punish huge errors. MSE is more accurate than MAE since it highlights large errors over small ones. MSE is also differentiable, so it can help to find maximums and minimums more effectively but by squaring all the values some errors are extremized. We can use RMSE to reduce the impact of this last point.

- MAE →when outliers are low
- MSE →when we have many outliers but not a lot of noise
- RMSE →if our large errors affect a lot the model's performance

3.1.2 Performance measures for classification

- **Accuracy**

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where:

  - TP = true positives
  - TN = true negatives

- FP = false positives
- FN = false negatives
- $TP + TN + FP + FN$ = total number of predictions

- **Precision**

$$Precision = \frac{TP}{TP + FP}$$

where:

- TP = true positives
- FP = false positives
- $TP + FP$ = total predicted positive instances

- **Recall**

$$Recall = \frac{TP}{TP + FN}$$

where:

- TP = true positives
- FN = false negatives
- $TP + FN$ = total actual positive instances

- **F1-Measure**

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

where:

- $Precision$ = proportion of correctly predicted positive instances
- $Recall$ = proportion of correctly identified actual positive instances
- $F1$ = harmonic mean of Precision and Recall

- **Cohen's Kappa Measure**

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

where:

- κ = Cohen's Kappa coefficient
- p_o = observed agreement between predictions and true labels
- p_e = expected agreement by chance

- **ROC Curve**

$$TPR = \frac{TP}{TP + FN} \qquad FPR = \frac{FP}{FP + TN}$$

where:

- TPR = true positive rate (Recall)
- FPR = false positive rate
- TP = true positives
- FP = false positives
- TN = true negatives
- FN = false negatives
- ROC curve = graphical plot of TPR against FPR

- **Area Under the Curve (AUC)**

$$AUC = \int_0^1 TPR(FPR)\, d(FPR)$$

where:

- AUC = area under the ROC curve
- TPR = true positive rate
- FPR = false positive rate
- higher AUC indicates better classifier performance

3.1.3 Features

Our data are characterized by their meaning, in this case we call those meanings features. Features can be useful if they help us to make a difference in our decision. An appropriate choice of the features is crucial for our models. Redundancy can slow down our models, so we want to cut the number of features as much as possible while retaining the biggest amount of information. We have many ways to do feature selection, and we usually do it in the pre-processing phase.

3.1.4 Experimental ML study

Since we can't know a priori the best model possible for our task we have to do some trial and error experiments. We should check as many algorithms as possible to be sure that our search space was covered enough.

The general pipeline of a machine learning study follows these steps

- Data preprocessing
- Choice of the method and the parameters
- Estimating the predictive error
- Inducing the final model
- Testing the final model

When we split the data we need to remember that we shouldn't do anything on the test set if we learn information from it (data leakage). If we don't have a naturally given test set we should get it from the learning one before doing any transformation.

The typical transformations include normalization, errors cleaning, outliers removal, missing values imputation and feature engineering. Remember to base the transformations to the knowledge present in the training data only. Normalization of the test set for example should be done only with parameters learned the training set. Anyway we should know the expected error of our model.

We can also get the validation set to find the best possible parameters and to assess overfitting. One way to have a more general partition of our data is obtained by using the so-called cross-validation

3.2 Genetic programming

Genetic algorithms are inspired from natures, they are not the only ones since we also have ANNs, particle swarm intelligence, fuzzy systems and evolutionary algorithms.

3.2.1 Evolutionary algorithms

We have two big hypotheses

- We have a problem that has a huge amount of possible solutions, among which we want to find the best one
- For each existing solution we are able to quantify its quality by means of a function called fitness (usually we associate a real-valued number to each solution)

The evolutionary process starts from a population to which we select some parents (ability of adapting, competition). These parents will then reproduce with some sort of variation (given by our genetic operators, usually crossover and mutation are applied). After this we will have our offspring population that will go into our original population (survival selection). At the reaching of a certain stopping condition we will select our final solution, the one with the highest fitness.

The two most common forms of evolutionary algorithms are

- Genetic algorithms →Our individuals are strings of constant length, and we usually try to use them for optimization
- Genetic programming →Individuals are computer programs, we can use this approach for machine learning.

The selection mechanism is similar since for both cases is done using fitness values. So we have things like fitness proportionate selection, ranking selection and tournament selection. The simplest one is fitness proportionate selection that takes the name of roulette wheel.

In genetic programming our representation is done by using trees. We can express a mathematical expression with this data-structure. We can define a tree using internal (F) and terminal nodes (T). We have many more ways.

Our fitness is a measure of the deviation between our expected target and the calculated output. So for regression we have RMSE, MAE or correlation while for classification we can have accuracy or F1.

With trees crossover can be done by moving around branches into the other tree. Mutation can be done by adding a branch randomly.

Genetic programming pseudocode

Algorithm 1 Genetic Programming Algorithm

1: 1. Initialize a population P of N programs (individuals) (typically at random)
2: 2. Repeat until termination condition:
3: 2.1 Execute each program in P and assign it a fitness according to how well it solves the problem
4: 2.2 Create an empty population P'
5: 2.3 Repeat until P' contains N individuals:
6: 2.3.1 Choose a genetic operator among crossover (with probability p_c), mutation (with probability p_m) and replication (with probability p_r). Remark that $p_c + p_m + p_r = 1$
7: 2.3.2 If the operator chosen in 2.3.1 was either mutation or replication, then select one individual from P (using one of the selection algorithms), apply the operator and insert the resulting individual in P'
8: 2.3.3 If the operator chosen in 2.3.1 was crossover, then select two individuals from P (using one of the selection algorithms), apply the operator and insert the resulting individuals in P'
9: 2.4 Selection of survivors: Extract one new current population P from P and P' (typically $P := P'$)
10: 3. Return the best individual in P

3.2.2 Symbolic regression

Symbolic regression is a task for which given a set of input-output pairs I_i, O_i we want to find (or approximate) a function that maps those points ($f(I_i) = O_i$) for any i.

Given a matrix with 2 input columns and 1 output column we have a regression in 2 variables. We will apply the following operators $+, -, *, //$ randomly in a tree and run our data. By calculating our fitness we have our first idea of how good our solution is. After building our population we will find the fitness of each point, we will select the parents with the roulette wheel, crossover and mutation will be applied, and our new population is there. We will re-run the algorithm until our stopping condition is met.

3.2.3 Why do evolutionary algorithms work?

Let

- s_t be the best individual in the population at generation t
- S_O be the set of all globally optimal solutions in the search space

$$\lim_{t \to \infty} P(s_t \in S_O) = 1$$

3.2.4 Peculiarities of genetic programming

The initialization of the population is the first step of the evolution process. It consists in the creation of the program structures that will later be evolved. The most common initialization methods in tree-based GP are

- Grow
- Full
- Ramped half-and-half

Grow When the "Grow" method is employed each tree of the initial population is built using the following algorithm

- A random symbol is selected with uniform probability from $\mathcal{F}$ to be the root of the tree.
- Let n be the arity of the selected function symbol. Then n nodes are selected with uniform probability from the set $\mathcal{F} \cup \mathcal{T}$ to be its sons
- For each function symbol between these n nodes, the method is recursively applied (its sons are selected from the set $\mathcal{F} \cup \mathcal{T}$), unless this symbol has a depth equal to d - 1. In the latter case its sons are selected from $\mathcal{T}$

Basically we avoid having single node trees at the starting point. Nodes with depths between 1 and d - 1 are selected with uniform probability from $\mathcal{F} \cup \mathcal{T}$, but once a branch contains a terminal node, that branch has ended, even if the maximum depth d has not been reached. Finally, nodes at depth d are chosen with uniform probability from $\mathcal{T}$. Since the incidence of choosing terminals from $\mathcal{F} \cup \mathcal{T}$ is random throughout initialization, trees initialized using this method are likely to have irregular shape (branches of various different shape).

Full Initialization

Instead of selecting nodes from $\mathcal{F} \cup \mathcal{T}$, the full method chooses only function symbols until a node is at the maximum depth. Then it chooses only terminals. The resulting tree has maximum depth for every branch.

Ramped Half-and-Half

Populations initialized with the above two methods might be built with trees that are too similar between them. In genetic programming diversity is really important, then to enhance the diversity of the population we can use this other technique. We start from d (maximum depth parameter) and the population is divided equally among individuals to be initialized with trees having depths equal to 1, 2, ..., d - 1, d. For each depth group, half of the trees are initialized with the full technique and half with the grow one.

Parameters

Once the set of functions and terminals are set, and we have the exact implementation for the genetic operators we have to select the parameters to use. Some are:

- Population size
- Stopping criterion
- Initialization technique
- Selection algorithm
- Crossover type and rate
- Mutation type and rate
- Maximum tree depth
- Presence or absence of steady state
- Presence or absence of ADFs or other modular structures
- Presence of absence of elitism

3.2.5 Weak points of genetic programming

We still have some problems that might arise

- Bloat →solvable with dynamic limits, tarpeian method or operator equalization
- Premature convergence (loss of diversity) →solvable with early stop/restart or with parallel and distributed genetic programming
- Overfitting →solvable with early stop/restart or multi-objective optimization

We can incorporate genetic programming into machine learning pipelines for our algorithms.

3.3 Geometric semantic genetic programming

Like Pokémon, we have evolutions with new operators. New crossovers and mutations that supposedly are better.

3.3.1 Optimization problems

Informally solving an optimization problem means to find the best solution(s) in a typically huge set of other candidate solutions. Formally a pair (S, f) where S is the set of all possible solutions (search space) and f is a function such that f = S $\rightarrow\mathbb{R}$. This function f quantifies the quality of the solutions in S and is called cost or fitness function.

An optimization problem is a minimization problem if it consists in looking for a solution o $\in$ S such that $f(o) \leq f(i), \forall i \in S$.
An optimization problem is a maximization one if it consists in looking for a solution o $\in$ S such that $f(o) \geq f(i), \forall i \in S$.
Such a solution is called global optimum (minimum or maximum).

The most natural and immediate method to solve an optimization problem is Hill climbing. It consists in trying to improve fitness step by step (stepwise improvement) by means of the concept of neighborhood. A solution $j \in S$ is called local optimum if

- for minimization problems $f(j) \leq f(i) \forall i \in N_j$
- for maximization problems $f(j) \geq f(i) \forall i \in N_j$

With hill climbing we will always have a local optimum that is not necessarily the global one.

3.3.2 Fitness Landscapes

If we want to represent the evolution of our fitness with respect to our solutions we can plot the fitness landscape. Formally defined as

- The fitness landscape can be represented as (S, V, f)
- S: set of potential solutions
- V: S $\rightarrow 2^S$, neighborhood function
- f: S $\rightarrow\mathbb{R}$, fitness function

Given our neighborhood function $\forall x \in S$

- $V(x) = \{y \in S | y = op(x)\}$
- $V(x) = \{y \in S | d(x, y) \leq 1\}$

The fitness landscape will give us a visual intuition of the facility or difficulty of a search agent to find the global optimum. A smooth landscape will imply an easy problem, while a rugged one will have many local optima and so a harder problem. In reality is impossible to draw a fitness landscape since our search spaces and neighborhoods will be huge.

The operator that is related to the neighborhood structure is called ball mutation. If we have a continuous fitness with a known optimum we have a CONO (continuous optimization with notorious optimum).

3.3.3 Genetic algorithms

Genetic algorithms are related to solutions where our individuals are strings of prefixed length. We can apply the usual crossover and mutation. We are not obliged to work with bits, but we can also use vectors of continuous values. We have many operators like Geometric ones. Geometric crossover for example differs from the normal

one. The coordinates of the offspring are the weighted average of the corresponding coordinates of the parents. In this case the offspring can't be worse than the parents.

Mutation is implemented by applying ball mutation, we move our points inside a radius.

3.3.4 Genetic programming

If in our evolutionary algorithms the individuals are computer programs we entered the setting of genetic programming. One example is Symbolic Regression where given a matrix we try to approximate the function that built it.

Genetic programming can be used as a machine learning method.

- Known: the correct output for a fixed given set of inputs $\{I_i, O_i\}$.
- Sought: a function belonging to a certain class that interpolates those points, so $f(I_i) = O_i \forall i$
- Output vector: the vector of the outputs of f is f(I) = f(I_i)
- Fitness: a measure of the error on the training set. Basically the distance between the output vectors of f and the target output vector $F(f) = D(f(I), O)$

We can apply this in a context of supervised learning.

3.3.5 Implementation of geometric semantic operators

We can save genetic information by applying the concept of semantics, two programs can have a very different structure but if the produced results are always the same they share the semantics.

Let $X = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n]$ be the set of input data (fitness cases) of a supervised learning problem and $\mathbf{t} = [t_1, t_2, \ldots, t_n]$ the vector of the respective expected output or target values.

A GP individual (or program) P can be seen as a function that, for each input vector $\mathbf{x}_i$, returns the scalar value $P(\mathbf{x}_i)$.

We define the semantics of an individual P as the vector

$$\mathbf{s}_P = [P(\mathbf{x}_1), P(\mathbf{x}_2), \ldots, P(\mathbf{x}_n)]$$

This is a point in an n-dimensional space.

We can map our programs into a semantic space (output) and visualize how far from the target they are. Traditional genetic operators will produce offspring by blindly manipulating the parents without caring about the semantics. We want to preserve this syntactic genetic material and define transformations that make sense from a semantic point of view.

If we are able to find a transformation on the syntax of the individual whose effect is ball mutation on the semantic space we induce an unimodal fitness landscape on every problem consisting in matching input data into known targets. Genetic programming should have a good evolvability on those problems.

3.3.6 Discussion

Ongoing research is being done to create the aforementioned operators

- Geometric semantic crossover →The only offspring generated by this crossover has a semantic vector that is a linear combination of the semantics of the parents with random coefficients included in [0, 1] and whose sum is equal to 1.
- Geometric semantic mutation →Each element of the semantic vector of the offspring is a weak perturbation of the corresponding element in the parent's semantic. We can tune its importance. It's basically a ball mutation in the semantic space, so we induce an unimodal fitness landscape.

These semantic operators can have some drawbacks, for example at each generation the offspring will be larger than the parents causing a fast growth in sizes. This makes the basically useless. One solution can be simplifying these individuals during the evolution.

3.3.7 Open issues

We still have some issues, one of them is that reconstructing the best individual can be impossible after too many generations. Also, these models are basically black boxes.

3.4 Semantic Learning algorithm based on inflate and deflate mutations

We left the last class with one problem, the genetic operators always create offspring bigger than their parents. Then our final model is huge and very difficult to interpret. We have now SLIM-GSGP (Semantic Learning algorithm based on Inflate and deflate Mutations).

We redefine Geometric Semantic Mutation as →given a parent function $T : R^n \to R$, GSM with mutation step ms returns the function $GSM(T) = T + \Delta T$. Where

- $\Delta T = ms(T_{R1} - T_{R2})$
- T_{R1} and T_{R2} are random functions.
- $\Delta T \in$[-ms, ms]

Note that the expression $T_{R1} - T_{R2}$ is used as a zero-centered random expression. T_{R1} and T_{R2} are normalized in the range [0, 1] applying a sigmoid. Then no crossover is needed in GSGP.

SLIM-GSGP follows two main steps

- If in the definition of GSM ΔT was subtracted, instead of added, the resulting operator would be equivalent to GSM.
- Given that the GSM uses + and the one that uses - are equivalent, we can alternate them without modifying the GSGP dynamics

The last step is called deflated mutation if the offspring is smaller than the parent. This is opposed to traditional GSM where we have inflated mutation. We can implement this by removing a term that was previously added.
Notice that

- Deflate mutation is a geometric semantic mutation
- Deflate mutation is not a backtracking operator

We need then to define a function with values in [-ms, ms] and one to obtain ball mutation on the semantic space.

Let $GSM(T) = T + \Delta T$, and S the sigmoid, then we can have the following ways of defining that ΔT. How to build a function in [-ms, ms]

1. $SIG2 = ms(S(T_{R1}) - S(T_{R2}))$
2. $ABS = ms(1 - \frac{2}{1+|T_R|})$
3. $SIG1 = ms(2S(T_R) - 1)$

How to get a ball mutation on semantic space

1. Summing a quantity close to 0, as in traditional GSM
2. Multiplying by a quantity close to 1, this can be done with
    - 1 + SIG2
    - 1 + ABS
    - 1 + SIG1

Then we have the following six variants of SLIM-GSGP

- SLIM + SIG2
- SLIM + ABS
- SLIM + SIG1
- SLIM * SIG2
- SLIM * ABS

- SLIM * SIG1

One very easy implementation of SLIM-GSGP can be done using a linked list.

The crossover can be implemented in different ways

- Swap crossover →swaps items of the linked list between parents, allowing recombination of genetic material while keeping offspring the same size as the parents
- Donor crossover →one parent donates a block to the other parent

These crossovers sometimes can enhance the predictive power of SLIM.

3.5 Neural networks

3.6 Introduction

ANNs are computational techniques that belong to the field of Machine Learning. The aim of these architectures is to mimic in a very simplified way the human brain. As we know the brain is composed by a set of simpler elements called neurons. The power of the brain is obtained by how these neurons interact and communicate between themselves.

Every neuron can do very simple computations but by combining a lot of them we can emulate more complex functions. The base structure in the field of neural networks is the so-called Perceptron

3.6.1 Perceptron

A Perceptron ca be considered the most basic neural network possible. By putting more of them together we can get more layer and so a multilayer Perceptron. One limitation of the perceptron is that the flow of information inside it is uni-directional (feed-forward neural network). The Perceptron is composed by one or more output neurons, each of which is connected to several input units by means of connections that are characterized by weights. In analogy with biology, these connections are called synapsis.

As we said the simplest structure is composed by just one neuron. The output of a perceptron can be calculated as

$$y = f(\sum_{i=1}^{n} w_i x_i + \theta)$$

The function f is called activation function. One of the most simple activations function is called Threshold function

$$f(s) \begin{cases} 1 & \text{if s} > 0 \\ -1 & \text{else} \end{cases}$$

This activation function is useful for classification problems. With this setup we can easily classify between 2 classes by projecting a line on a plane (binary classification). The Perceptron will classify one or the class based on the output that the function gives.

We should define two phases for our neural networks

- Learning →The network modifies the weights
- Generalization →We can use the network on new data for our purposes

During the learning phase the network receives the training data and with the use of certain algorithms it will change the weights to adjust the classification outputs. If we know our target we are doing obviously supervised learning.

The algorithm used by the perceptron to modify the weights is the following

1. Initialize the connections with a set of weights generated at random.
2. Select an input vector $\bar{x}$ from the training set. Let y be the output value returned by Perceptron for this input value and d the correct answer (target), that is associated to $\bar{x}$ in the dataset.
3. If $y \neq d$ (the calculated output is wrong), then the weights (w_i)of all connections are modified as follows

- if y > d $w_i := w_i - \eta x_i$
- if y < d $w_i := w_i + \eta x_i$

4. Back to 2

The algorithm terminates when all vectors in the training set are classified in a correct way or after a certain prefixed number of iterations. Remember that also the value of the threshold θ needs to be changed (but the input will always be 1). The parameter η is called learning rate and indicates by how much we need to adjust our changes, it can be fixed or variable (but always positive).

As we said we want to find a line that separates the 2 classes. The right term that will allow the generalization entering the realm of networks is hyperplane. With 2 inputs we have

$$w_1 x_1 + w_2 x_2 + \theta = 0 x_2 = -\frac{w_1}{w_2} x_1 - \frac{\theta}{w_2}$$

The straight line that we get allows us to graphically separate the two classes.
This straight line has the following properties

- All the points that belong to class 1 are "above" the line and all the ones of class 2 are "below"
- The weights vector is perpendicular to the separating line
- θ allows us to calculate the distance of the straight line to the origin

If a weight vector that allows us to obtain y = d for all training instances exists than the Perceptron will converge to a solution in a finite number of steps without caring about the initial choice of the weights. And if this vector exists then the classes are linearly separable.

If our data can be partitioned in more than two classes we are in the realm of multiclass classification. Given a problem in which data must be classified into more than two classes, more than two neurons are needed. We simply connect our input layer to an output layer made of a number of neurons equal to the number of classes. The Perceptron is able to separate data into 2^m classes depending on the binary output values of its output.

During training each single neuron is trained independently with the Perceptron learning rule. At the end then we will get a weight vector for each neuron. Using this vector we can define the needed hyperplane.

The class of problems that we saw is the one of linearly solvable ones. But if we have a XOR problem we can't really use a linear function since no set of straight line will be able to separate the two classes. Then for this particular problem we have to add a new layer that will correct the predictions. Our new network will have a hidden neuron between the input and the output layer (hidden layer). In order to solve non-linear problems we must use neural networks with at least one hidden neuron.

So far we only looked at neurons that use the threshold (step) function as activation. But we can have many more

- Logistic or sigmoid $f(x) = \frac{1}{1+e^{-ax}}$
- Hyperbolic tangent $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- Gaussian $f(x) = e^{-x^2}$

3.6.2 ADALINE

An evolution based on the normal perceptron is ADALINE, the difference is that the learning rule is called Delta Rule and can be seen as a generalization of the Perceptron learning rule. The difference is given on how the output is managed. The Perceptron uses as output the result of an activation function to learn while the Delta Rule uses the result of the summation to which it can, or can not, be applied an activation function, so:

$$y = \sum_{i=1}^{m} w_i x_i + \theta$$

or

$$y = f\left(\sum_{i=1}^{m} w_i x_i + \theta\right)$$

The main difference between Perceptron and Delta Rule is that the latter compares this calculated output with the desired output by using an error. The error function can be defined either as mean square error $E = \frac{1}{n}\sum_{i=1}^{n}(d_i - y_i)^2$ or as absolute error $E = \sum_{i=1}^{n}|d_i - y_i|$. The Delta Rule looks for the vector of weights that minimizes the error function using the method of gradient descent (then we arrive at an optimization problem). We can visualize this error as a surface where we look for the point that minimizes our function looking for the minimum.

If the problem is linearly separable then our surface is unimodal, and so we have only one global minimum and no local minimum. Delta Rule will try to minimize at each step the error. If the problem is not unimodal then ADALINE will probably fall into a local minimum.

To decrease the error E at each step the weights are modified by a quantity Δw which is proportional to the derivative of the function E $\rightarrow \Delta w = -\eta\frac{\partial E}{\partial w}$. The weights are modified in the negative direction of the gradient of E. At the minimum the derivative is 0 and so the algorithm stops.

- Initialize all connections with random weights;
- Repeat until (termination condition):
  - Select a training observation j
  - Calculate the output of the neuron for this observation:

$$y_j = f\left(\sum_{i=0}^{m} w_i x_{ji}\right)$$

  - If y_j approximates d_j in a satisfactory way, then do nothing, else, for each connection $i = 1, 2, \ldots, m$:
    - if $y_j < d_j$ then $w_i = w_i + \eta y_j(1 - y_j)x_{ji}$
    - else $w_i = w_i - \eta y_j(1 - y_j)x_{ji}$
- end repeat

As said we stop when we converge, but we can also decide a maximum number of iterations. Here we have an example of the derivative of the sigmoid

$$f(x) = \frac{1}{1 + \exp(-x)}$$

Its derivative $f'(x)$ can be calculated as follows:

$$\begin{aligned}
f'(x) &= \frac{\partial}{\partial x}\big(1 + \exp(-x)\big)^{-1} \\
&= -1 \cdot \big(1 + \exp(-x)\big)^{-2} \cdot \big(-\exp(-x)\big) \\
&= \frac{\exp(-x)}{\big(1 + \exp(-x)\big)^2} \\
&= \frac{1}{1 + \exp(-x)} \cdot \frac{\exp(-x)}{1 + \exp(-x)} \\
&= \frac{1}{1 + \exp(-x)} \cdot \left(\frac{1 + \exp(-x) - 1}{1 + \exp(-x)}\right) \\
&= \frac{1}{1 + \exp(-x)} \cdot \left(1 - \frac{1}{1 + \exp(-x)}\right) \\
&= f(x)\big(1 - f(x)\big)
\end{aligned}$$

Therefore, the derivative of the sigmoid can be expressed as a function of the sigmoid itself

3.6.3 Non-linearly separable problems

In general, non-linearly separable problems can be solved using multi-layer Neural Networks, i.e. Neural Networks in which one or more neurons are not directly linked to the output of the network (we call these neurons hidden neurons, and given that they are generally organized into layers, we call these layers hidden layers).

3.6.4 Backpropagation

Backpropagation is an algorithm that can be applied to a multi-layer perceptron in order to change its weights and optimize them. The main characteristics of a feed-forward neural network are:

- The neurons in a given layer receive as input the outputs of the neurons of the previous level
- There are no inter-level connections
- All the possible intra-level connections exist between two any subsequent layers

The Backpropagation is the most common learning rule for multi-layer Neural Networks (sometimes also called multi-layer Perceptron or Feed-Forward Neural Networks). The Backpropagation can be seen as a generalization of the Delta-Rule to multi-layer Neural Networks. The Delta-Rule modifies the weights of each neuron on the basis of the error committed by that neuron. To be able to extend the Delta-Rule to multi-layer networks is how to modify the weights of the neurons that belong to the hidden layers. In fact, the correct output for these neurons is not known, and thus we cannot directly calculate the error.

The basic idea of the Backpropagation is the following: the errors of the output neurons are propagated backwards to the hidden neurons. In other words: the error of a hidden neuron is defined as the sum of all the errors of all the output neurons to which it is directly connected.

The Backpropagation can be applied to networks with any number of layers, but the following theorem holds: Universal Approximation Theorem (Hartman, Keeler, Kowalski, 1990) Only one level of hidden neurons is sufficient to approximate any function (with a finite number of discontinuities) with arbitrary precision, provided that the activation functions of the hidden neurons are non-linear. This is one of the reasons why one of the most used configurations of a Feed-Forward multi-layer Neural Network is with only one layer of hidden units that use a sigmoidal activation function

The Backpropagation is composed by one forward step and one backward step. In the forward step, the weights of the connections remain unchanged and the outputs of the network are calculated propagating the inputs through all the neurons of the network. At this point, the error of each output neuron is calculated. The backward step consists in the modification of the weights of each connection. This calculation is made in the following way:

- For the output neurons, the modification is done by means of the Delta Rule
- for the hidden neurons, the modification is done by propagating backwards the error of the output neurons: the error of each hidden neuron is considered as being equal to the sum of all the errors of the neurons of the subsequent layer.

The weights of the connection that enter each neuron are updated using the formula:

$$w_{ij} = w_{ij} + \Delta w_{ij}$$

where w_{ij} is the weight of the connection between neuron i and neuron j, and:

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$$

Now the problem is how to calculate Δw_{ij} without having to calculate every time the derivative of the error function E.

1. Initialize all the weights in the network with random values.
2. Select a vector $\mathbf{x}$ of the training set (let $\mathbf{d}$ be the correct output – or target – that can be found in the dataset, corresponding to vector $\mathbf{x}$). Calculate the output of the network $\mathbf{y}$ for input $\mathbf{x}$.
3. If $\mathbf{y}$ is satisfactory (i.e., $\mathbf{y}$ approximates $\mathbf{d}$ in a sufficient way), then **go to** step 2. Else:

   3.1. For each output neuron j, calculate:

   $$\delta_j = (d_j - y_j) \cdot y_j \cdot (1 - y_j)$$

   and modify the weight of the connections that enter into neuron j as follows:

   $$w_{ij} = w_{ij} + \eta \delta_j y_i$$

3.2. For each hidden neuron j, calculate:

$$\delta_j = y_j(1 - y_j) \sum_k \delta_k w_{jk}$$

(where the sum is calculated over all output neurons k), and modify the weight of the connections that enter into neuron j as follows:

$$w_{ij} = w_{ij} + \eta \delta_j y_i$$

(In both cases, y_i is the value entering in neuron j from source i)

4. **go to** step 2.

The algorithm terminates when one of the following conditions is satisfied:

- The correct outputs **d** are approximated from the outputs **y** of the network in a "satisfactory" way for every vector **x** of the training set.
- A prefixed number of maximum iterations has been executed.

As for other algorithms we can improve Backpropagation by selecting the right kind of parameters, the main ones are

- Learning rate η →if is too small the learning will be slow, if is too big the network will be unstable
- Momentum α →we can use it to reduce the risk of staying too much inside a local minimum
- Number of hidden neurons →Usually based on experience, we risk to fall inside a local minimum

To have a good result for our network we have to avoid both underfitting and overfitting. We can find the sweet spot by selecting an appropriate size for our network.

3.6.5 Cyclic (or Recursive) Neural networks

An improvement on normal feed-forward networks can be obtained by using cyclic neural networks. Some example can be Jordan, Elman, Hopfield and Bolzman machines. In Jordan networks we have a state unit that helps the learning phase by combining input and state units. Backpropagation is the learning rule.

3.7 Neuroevolution and NEAT

We call Neuroevolution the artificial evolution of neural networks using evolutionary algorithms.

4 Text mining

4.1 Introduction to NLP

Natural language processing is an area of computer science and artificial intelligence that is concerned with the interaction between computers and humans in natural language. The ultimate goal of NLP is to enable computers to understand language as well as we do.

Some applications of NLP can be machine translation, dialog systems, search engines and predictive keyboards with autocorrectors.

4.1.1 Challenges of natural language

Natural language is made of two main elements:

1. Grammatical system, with its own rules, used by people to communicate
2. Natural evolution due to people communication (like new words or regularization of the language)

Some problems in its analysis can arise from the variability of it. Different sentences can have the same meaning (paraphrases) or a single sentence can have multiple meanings (ambiguity). NLP systems should also be able to generalize as much as possible (work in different languages), to achieve this we train it on a corpus. Since the system can encounter inputs that it never encountered (Out-of-Domain or Out-of-Vocabulary) we want to extend its knowledge to these new words or meanings.

4.1.2 The NLP pipeline

A typical NLP pipeline follows these steps: Corpus →Feature engineering →Task.

- **Corpus** →collection of text organized into datasets. It can be made up of everything that is written in natural language. A collection of more than one corpus is called corpora. First we split the corpus into train, validation and test set (or we can use k-fold cross-validation)
- **Feature Engineering** →we need a way to represent text in a way that a computer can understand it. The most basic way is the Bag-of-Words representation (glorified 1-Hot-encoding), it gives us a sparse representation of our documents.
- **Task** →we can have multiple tasks like keyword extraction, text similarity, sentiment analysis and many more. Given two documents we can transform them into sparse vectors and compare them with simple distance metrics. If they share a lot of words they will be close to each other. This is the basic for multiple tasks. We can use euclidean distance ($D(a,b) = \sqrt{\sum_{i=1}^{n}(b_i - a_i)^2}$) to calculate the distance matrix and apply KNN to label our documents.

4.1.3 Text preprocessing methods

Before applying BoW we need to define what's a word, this process is called **Tokenization**. We want to identify tokens (features in text), but it's not a trivial task. Some problems can arise from composed words and punctuation. It's also clear that with this kind of sparse representation we can encounter problems like the usual curse of dimensionality and the fact that we lose semantic relationships.

- Curse of dimensionality →The total dimension of the BoW is the vocabulary size. Our model can easily overfit. We can use dimensionality reduction techniques.
- **No semantic relationships** →BoW doesn't consider the semantic relationships between words since we discard the order and similar words can be considered as different features.

Following Zipf's law we know that some words have not a very big importance, but they are really common (the so-called stop words). Stop words presence means that we have more rules and more processing to apply.

We should also lowercase everything to reduce the vocabulary size (but take care of names) and normalize dates, numbers and names by either using regular expressions or applying a default token in their place.

We can also transform words in a way that if they represent the same meaning they are captured by the same token. This can reduce language inflection. Some ways are

- Stemming →Remove the last characters to obtain a shorter form even if it has no meaning.

- Lemmatization →Turns the words into their root word

We can also just consider certain type of words like verbs, nouns, adjectives and adverbs.

4.2 Word representation

4.2.1 Classification with BoW

We just throw our binary vectors inside a neural network.

4.2.2 Word representations

BoW is the baseline for every text mining task since it has a lot of limitations like curse of dimensionality, no semantic relationship, all words will have the same importance and the context is ignored even though it changes the meaning.

To solve these problems we want to use vectors to represent our words. From words to tokens to embeddings. Using embeddings for words is better than embed full documents since we can have a more granular and precise representation. One of the breakpoints was the creation of Word2Vec in 2013.

We want that our word representations will be able to capture semantic relationships in a mathematical way. Some ways to generate our word vectors are

- Term-Term matrix
- Point-Wise mutual information (PMI)
- Skip-gram(Word2Vec)

In more details

Term-Term matrix

Term-term matrix is of dimensionality |V|x|V| and each cell records the number of times the row word (target) and the column word (context) co-occur in the training corpus

	data	model	learning	algorithm
data	25	12	9	7
model	12	30	15	18
learning	9	15	28	14
algorithm	7	18	14	26

PMI

Point-wise Mutual Information (PMI) is a measure of how often two events x and y occur, compared with what we would expect if they were independent:

$$I(x,y) = \log_2 \frac{P(x,y)}{P(x)P(y)}$$

The point-wise mutual information between a target word w and a context word c is then defined as

$$PMI(w,c) = \log_2 \frac{P(w,c)}{P(w)P(c)}$$

It is common to use Positive PMI (called PPMI) which replaces all negative PMI values with zero

$$PPMI(w,c) = max(\log_2 \frac{P(w,c)}{P(w),P(c)}, 0)$$

To build our PMI between information and data

- Calculate co-occurence matrix
- Calculate word probability
- Calculate co-occurence probability

- Create probabilities matrix
- Calculate PPMI

With this approach we solve the problems of semantic relationships and the one of the words having the same importance. We still have the curse of dimensionality and the fact that context is not accounted for.

4.2.3 Vector embeddings (Word2Vec)

We can use another strategy to solve also the curse of dimensionality problem. Using an approach called Skip-gram (or Word2Vec) we can create short and dense word vectors. This approach works in the following way

1. We start from the idea that similar words will tend to appear together
2. We train a classifier, a simple neural network with one hidden layer, to perform a binary prediction of the kind →Is word w likely to show up near c?
3. We learn the weights of the hidden layer, aka the word vectors that we are trying to learn

More in details

1. Define word and context, for each word in our corpus select the context window
2. Add negative samples for words that do not appear in the same context window
3. Define the task, usually if word w and c appeared in the corpus
4. Train the model
5. Retrieve the embeddings

After this pipeline we have the possibility to extract semantic relationships between words

4.2.4 Classification with Word2Vec

To make predictions using our embeddings we need to find a way to give the vectors as input. Some ideas can be to use the mean of the vector or apply an RNN since we consider the input as a varying one.

4.3 Sequential models

4.3.1 Review of word embeddings

From the previous class we know that some problems are present with Bow representation. To solve them we can use Word2Vec, but we still have the problem of having to input a sequence inside our neural network. We can average our vectors into one scalar, but we will lose a lot of semantic information. To solve this problem we can use recurrent neural networks (RNNs).

4.3.2 RNNs

To process sequential input we can use some models like RNNs and LSTMs. Some common tasks can be next word prediction, sequence labeling, sequence classification and machine translation.

A recurrent neural network is a network that contains a cycle within its network connections, meaning that the value of some unit is directly, or indirectly, dependent on its own earlier outputs as an input.
In a standard RNN architecture, three main matrices are involved:

- U: the input-to-hidden weight matrix, responsible for processing the current input.
- W: the hidden-to-hidden recurrent weight matrix, which propagates information from the previous hidden state to the current one.
- V: the hidden-to-output weight matrix, used to generate the network output.

These matrices are shared across all time steps, meaning that the same parameters are reused throughout the sequence. This parameter sharing allows the network to generalize across sequences of different lengths and reduces the total number of trainable parameters.

At each time step t, the network computes:

$$h_t = f(Ux_t + Wh_{t-1})$$

where:

- x_t is the input vector at time step t,
- h_{t-1} is the hidden state from the previous time step,
- h_t is the updated hidden state,
- $f(\cdot)$ is a nonlinear activation function such as tanh or ReLU.

The output of the network is then computed as:

$$y_t = g(Vh_t)$$

where:

- y_t is the output at time step t,
- $g(\cdot)$ is typically a softmax or another activation function depending on the task.

Because the hidden state carries information across time, RNNs are particularly useful for sequential learning problems where context and order are important.

For a sentence/document, each word (embedding) is processed through the network one step at the time. In parallel, the hidden vector of the previous word, moves in the network and is used to calculate the hidden vector of the next word. The main problem with RNNs is related to long dependencies, The final hidden state reflects more information about the end of the sentence than it beginning.

Another problem is the one of vanishing gradient, during the backward pass of training, the hidden layers are subject to repeated multiplications, as determined by the length of the sequence. A frequent result of this process is that the gradients are eventually driven to zero, which means that the initial weights cannot be updated properly.

The long dependencies' problem can be solved by using bidirectional models.

4.3.3 LSTM

As said we can use bidirectional models to solve some problems related to RNNs. LSTMs models are explicitly designed to avoid the long term dependency problems. The context management is divided into two subproblems:

- Removing information no longer needed from the context
- Adding information likely to be needed for later decision-making

To accomplish this LSTMs use:

- Context layer (cell state)
- A specialized neural unit that work as gates to control the flow of information into and out of the units that comprise the network layers.

The cell state is the horizontal line that runs through the top of the diagram. It acts as a transport highway that transfers relative information all the way down the sequence chain. Is like the memory of the network.

The main gates in LSTMs are

- **Forget gate** →This gate decides what information should be thrown away or kept in the cell state. Information from the previous hidden state and information from the current input is passed through the sigmoid function. Values come out between 0 and 1. The closer to 0 means to forget, and the closer to 1 means to keep.

- **Input gate** →This gate decides what information should be used to update the cell state: first, the previous hidden state and the current input are passed through a sigmoid function, which produces values between 0 and 1 to determine which information is important to keep; simultaneously, the same inputs are passed through a tanh activation function to squash values between -1 and 1, helping regulate the network; finally, the outputs of the sigmoid and tanh functions are multiplied together so that the sigmoid gate controls which information from the tanh output should be retained. Next, the cell state is pointwise multiplied by the forget vector, allowing the network to discard information associated with values close to 0; afterward, the output of the input gate is added pointwise to the cell state, updating it with new information considered relevant by the neural network.
- **Output gate** →This gate determines the next hidden state or, at the final time step, the prediction of the network: first, the previous hidden state and the current input are passed through a sigmoid activation function; then, the updated cell state is passed through a tanh function; afterward, the outputs of the sigmoid and tanh functions are multiplied together to determine which information should be carried forward in the hidden state; the resulting vector becomes the new hidden state, while both the updated cell state and hidden state are propagated to the next time step or forwarded to an output activation layer to produce the final prediction.

4.3.4 Seq-to-seq models

We already cited the task of machine translation, but we also have another one that enters the realm of sequence to sequence, chatbots.

Sequence-to-sequence (Seq2Seq) models are a class of neural network architectures designed to transform one sequence into another sequence. They are widely used in tasks such as machine translation, text summarization, speech recognition, and question answering.

In Seq2Seq models, both the input and the output are sequences of variable length. For example, in a machine translation task, the input sequence may be a sentence in English, while the output sequence is the translated sentence in another language.

A Seq2Seq architecture is generally composed of two main components:

- An **encoder**,
- A **decoder**.

The encoder processes the input sequence and converts it into a fixed-length vector representation commonly called the **context vector**. This vector summarizes the semantic information contained in the entire input sequence.

The encoder is typically implemented using a recurrent neural network (RNN), such as a Long Short-Term Memory (LSTM) network or a Gated Recurrent Unit (GRU). At each time step, the encoder receives an input token and updates its hidden state according to the sequential information processed so far.

Given an input sequence:

$$x_1, x_2, x_3, \ldots, x_T$$

the encoder updates its hidden state recursively as:

$$h_t = f(x_t, h_{t-1})$$

where:

- x_t is the input token at time step t,
- h_{t-1} is the previous hidden state,
- h_t is the current hidden state.

After the final input token has been processed, the last hidden state of the encoder is used as the context vector:

$$c = h_T$$

This context vector is then passed to the decoder as its initial hidden state.

The decoder is another recurrent neural network responsible for generating the output sequence one element at a time. At each time step, the decoder receives:

- The previous output token,
- The previous hidden state,
- The context vector.

The decoder hidden state is computed as:

$$s_t = f(y_{t-1}, s_{t-1}, c)$$

where:

- y_{t-1} is the previously generated output token,
- s_{t-1} is the previous decoder hidden state,
- c is the context vector generated by the encoder.

The decoder output is obtained through a linear transformation followed by a softmax activation function:

$$y_t = \text{softmax}(W s_t)$$

Where W is the output weight matrix.

The softmax function converts the decoder output into a probability distribution over the vocabulary. The token with the highest probability is selected as the generated word for the current time step.

During training, the decoder often receives the true previous target word as input, a technique known as **teacher forcing**. During inference, instead, the decoder uses the word generated at the previous time step.

As an example, consider the sentence:

"Great movie"

which must be translated into Portuguese:

"Ótimo filme"

The encoder first converts the input words into embedding vectors:

$$\text{great} \rightarrow [0.03,\ 0.98]$$

$$\text{movie} \rightarrow [0.76,\ 0.85]$$

The encoder processes these embeddings sequentially and generates the context vector. The decoder then uses this context vector to produce the translated sequence token by token:

$$< s > \rightarrow \text{ótimo} \rightarrow \text{filme} \rightarrow < /s >$$

At every decoding step, the softmax layer produces a probability distribution over all words in the vocabulary, allowing the model to determine the most likely next word.

Seq2Seq architectures represented one of the first successful approaches for neural machine translation and sequence generation tasks. However, traditional Seq2Seq models may struggle with long sequences because the entire input information must be compressed into a single context vector. This limitation motivated the development of attention mechanisms and Transformer architectures.

4.4 Attention and transformers

4.4.1 Attention mechanisms

In the previous class we saw how RNNs work but with modern LLMs we need to add some elements. For example, we need to talk about attention mechanisms. As we know classical encoder-decoder methods have to encode all the information about a sentence inside a vector while the decoder must create the translation using only that vector.

The final hidden state is like a bottleneck where all the information is represented in a smaller representation with respect to the original. The decoder if trained well should be able to recreate the original input using only the context vector. One problem is that languages behave differently from the syntactic point of view, so the model should remember the semantic information as much as possible.

A way to solve this problem is using the mechanism of attention. We allow the decoder to get information from all the hidden states of the encoder, not just the last one. The first idea of attention is to create a fixed-length vector by taking a weighted sum of all the encoder hidden states. At each step, the decoder does an extra step before producing it's output. The pipeline is the following:

- Look at the sets of hidden states, each encoder hidden state is most associated with a certain word in the input sentence
- Give to each hidden state a score
- Multiply each hidden state by its softmaxed score, thus amplifying hidden states with high scores, and drowning out hidden states with low scores

Add formulas

4.4.2 Transformer architecture

Some stories on GPU and why generating cat videos is making a shitty 12mb GPU cost like 9000 dollars. Anyway we can't use GPUs for RNNs because parallelization is basically impossible. To solve this problem Self-attention was created.

Self-attention allows each token to directly attend to all other tokens in the sequence at the same time, capturing dependencies regardless of distance. Here we have the first important building block of transformers.

The standard architecture for building modern large language models is the Transformer. Like LSTMs, Transformers can capture long-distance relationships between words in a sequence. However, unlike LSTMs, Transformers do not rely on recurrent connections that process tokens step by step. Instead, Transformers use self-attention, a mechanism that allows each token to directly attend to all other tokens in the sequence. This makes it possible to compute token representations in parallel, improving training efficiency and making much better use of GPUs.

Transformers are made of two main types of blocks: encoders and decoders. The Encoder reads the input sequence and transforms each token into a contextual representation. The Decoder uses these representations to generate the output sequence, one token at a time. In the original Transformer architecture, the encoder and decoder work together: the encoder represents the input, and the decoder produces the output based on both the previous generated tokens and the encoder's representations.

Each transformer will have many encoders and many decoders Inside each Encoder, we will find a layer of self-attention and a standard Neural Network - information moves forward through the encoders Decoders receive the output from the Encoder and move them through two attention layers (self-attention and encoder-decoder attention) and a (feed-forward) neural network. The last decoder layer produces an output.

4.4.3 Transformer encoders

Some Transformers use only the encoder, because the goal is to understand text rather than generate new text. Encoder-only models read the full sentence at once, capturing context from both previous and following words. They produce rich, context-aware word and sentence representations, usually stronger than static embeddings like Word2Vec or GloVe. These representations can be reused in downstream tasks such as classification, sentiment analysis, NER, similarity, clustering, and retrieval (Transfer Learning).

BERT (Bidirectional Encoder Representations from Transformers), developed by Google, is probably the most famous Encoder-only Transformer model. BERT (BERTLarge) model uses 24 encoder layers, referred to in the paper as Transformer blocks. BERT uses 16 self-attention heads, allowing the model to capture multiple contextual relationships in parallel. It has a hidden size of 1024 units, making its internal representations larger and more expressive. This means that each token is represented by a 1024- dimensional contextual embedding produced by the final encoder layer. The representations generated by BERT can then be fed to another model as word embeddings

Transfer learning is the process of reusing knowledge learned from one task to improve performance on another task. In NLP, models such as BERT are first pre-trained on massive text corpora to learn general language patterns. They are called Pre-trained Language Models (PLMs) This models can be called Foundation models - very large pre-trained models trained on broad and diverse datasets at massive scale. The pre-trained model is then adapted (fine-tuned) to a specific downstream task using a smaller labeled dataset

BERT was pre-trained on large unlabeled datasets, including English Wikipedia and the Bookworm's collection of books. The model learns language patterns using self-supervised tasks such as Masked Language Modeling (MLM) and Next Sentence Prediction (NSP).

- In MLM, during training, random words are masked, and BERT learns to predict them using context from both directions of the sentence.
- In NSP, BERT receives two sentences as input and predicts whether the second sentence logically follows the first.

The original BERT Large model contains approximately 340 million parameters, with 24 encoder layers, 16 attention heads, and 1024 hidden dimensions. BERT training is a two-step process: first pre-training a model (which is already done when we use it) and then fine-tuning it for a particular task. Finally, we can fine-tune it for tasks like sentiment analysis or predicting movie reviews.

Word and Sentence Embeddings

Since BERT produces multiple representations for the same word across layers, we must choose how to extract the final embedding. To extract embeddings for each word instead of a sentence level one, we can aggregate information from all token embeddings using pooling strategies. Common approaches include:

- Last Layer: use the final hidden representation of each token.
- Sum all layers: sum the token representations from all Transformer layers.
- Sum last four: sum the token representations from the final four layers.
- Concat last four: join the final four token representations into one larger embedding vector.

BERT adds a special token called [CLS] at the beginning of every input sequence. Across multiple Transformer layers, the [CLS] embedding gradually becomes a contextual summary of the entire sequence. During pre-training objectives such as Next Sentence Prediction, but also during fine-tuning, we can explicitly force [CLS] to capture sentence-level information. After the final encoder layer, BERT outputs one contextual embedding per token. For sequence classification tasks, the final embedding corresponding to [CLS] is usually selected as the sentence representation. This vector is then fed into a simple classifier, typically a linear layer followed by Soft max or a Logistic Regression.

Classification and Variations

We can perform classification directly with a task-specific model or indirectly with general-purpose embeddings. A task-specific model is a representation model, such as BERT, trained for a specific task like sentiment analysis. Alternatively, BERT can be used to generate general-purpose context embeddings that can be used for a variety of tasks not limited to classification, like semantic search. Over the years, many variations of BERT have been developed, including:

- RoBERTa
- DistilBERT
- ALBERT
- DeBERTa

4.5 LLMs

4.5.1 Transformer architecture

We recall that deep learning relies a lot on GPUs since we can parallelize computation. However, RNNs process tokens one word at a time, with each hidden state depending on the previous one, so computation must proceed sequentially. This implies that GPUs cannot use their cores at full potential.

The sequential nature of attention based RNNs prevents parallelization during training of the model, making this process less efficient and not well-optimized. This issue was solved using the mechanism of self-attention. It allows each token to directly attend to all other tokens in the sequence at the same time, capturing dependencies

regardless of distance. The creator of self-attention proposed a new model called Transformer, that uses self-attention and Feed-forward networks eliminating sequential computation, and enabling fast and efficient training on modern hardware.

This is the standard architecture for building modern LLMs. Like LSTMs it can capture long-distance relationships between words in a sequence. But unlike LSTMs, transformers do not rely on recurrent connections that process tokens step by step. Using self-attention transformers can compute token representations in parallel, improving training efficiency and making much better use of GPUs.

Transformers are made of two main types of blocks

- Encoder →Reads the input sequence and transforms each token into a contextual representation
- Decoder →Uses representations to generate the output sequence, one toke at a time

In the original architecture they worked together.
Each transformer will have many encoders and decoders.

- Inside each encoder we have a layer of self-attention and a standard neural network, information moves forward through the encoders
- The decoders receive the output from the encoder and move them through two attention layers and a neural network. The last decoder layer produces an output. We might also find a layer of encoder-decoder attention.

4.5.2 Transform encoders

Some transformers use only the encoder, because the goal is to understand text rather than generate it. These models read the full sentence at once, capturing context from both previous and following words. With this they can produce context aware word and sentence representation. We can reuse them in downstream tasks in a context of transfer-learning.

Generating Word Representations with Transformers and Transformer Encoders introduces BERT (Bidirectional Encoder Representations from Transformers), developed by Google, which is probably the most famous Encoder-only Transformer model. The BERT (BERTLarge) model uses 24 encoder layers, referred to in the paper as Transformer blocks. BERT uses 16 self-attention heads, allowing the model to capture multiple contextual relationships in parallel. It has a hidden size of 1024 units, making its internal representations larger and more expressive. This means that each token is represented by a 1024-dimensional contextual embedding produced by the final encoder layer. The representations generated by BERT can then be fed to another model as word embeddings.

Regarding the training of BERT and Transformer Encoders, transfer learning is the process of reusing knowledge learned from one task to improve performance on another task. In NLP, models such as BERT are first pre-trained on massive text corpora to learn general language patterns. They are called Pre-trained Language Models (PLMs). This models can be called Foundation models - very large pre-trained models trained on broad and diverse datasets at massive scale. The pre-trained model is then adapted (fine-tuned) to a specific downstream task using a smaller labeled dataset.

Continuing with Training BERT and Transformer Encoders, BERT was pre-trained on large unlabeled datasets, including English Wikipedia and the BookCorpus collection of books. The model learns language patterns using self-supervised tasks such as Masked Language Modeling (MLM) and Next Sentence Prediction (NSP). In MLM, during training, random words are masked, and BERT learns to predict them using context from both directions of the sentence. In NSP, BERT receives two sentences as input and predicts whether the second sentence logically follows the first. The original BERT Large model contains approximately 340 million parameters, with 24 encoder layers, 16 attention heads, and 1024 hidden dimensions.

The stages of Training BERT and Transformer Encoders follow a specific training process:

1. First pre-training a model (which is already done when we use it)
2. And then fine-tuning it for a particular task (there are a lot of models already fine-tuned in hugging face for ex.)
3. Finally, we can fine-tune it for a particular task, like sentiment analysis or predicting movie reviews

In the context of Training BERT and Transformer Encoders, over the years, many variations of BERT have been developed, including RoBERTa, DistilBERT, ALBERT, and XLM-RoBERTa, each trained in various contexts.

4.5.3 Classification with encoders

To extract embeddings for each word instead of a sentence level one we can aggregate information from all token embeddings using pooling strategies like:

- Last layer
- Sum all layers
- Sum last four
- Concat last four

BERT adds a special token called [CLS] at the beginning of every input sequence. This token during self-attention can attend to all words in the sentence to help the making the context. After it BERT will output one contextual embedding per token. This will then be sent into a simple classifier.

We can perform classification directly with a task specific model or indirectly with a general-purpose one. In this setting a task-specific model is one model that is built to support one specific work, like sentiment analysis. The embeddings can be used also for semantic search.

To use BERT we have to

- Tokenize the input sequence
- Embed our sentences
- Use the embeddings as features
- Train a classification model

4.5.4 Self-attention

Recall that like LSTMs, transformers can capture long-distance relationships between words in a sequence. However, the work for the seconds use self-attention and not recurrent connections. This mechanism will allow each token to directly attend to all other tokens in the sequence.

Self- attention is different from attention, first is made of two steps

- Relevance scoring for each position
- Combine the information based on those scores

In the encoder we have to generate as always a hidden state, using self attention we have to consider 3 main elements

- Query →current focus of attention
- Key →context input being compared to the current focus
- Value →used to compute the output for the current focus of attention

The three of them have different roles, to capture them we use weight matrices to project each input vector x_i into a representation of its role as a key (K), query(Q) or a value (V).

- $q_i = W^Q x_i$
- $k_i = W^K x_i$
- $v_i = W^V x_i$

Self attention is carried out by the interaction of these matrices, that are produced by multiplying the input layer with the projection matrices. The scoring of the relevance is done by multiplying the query associated with the current position with the key matrix. Self attention combines the relevant information of previous positions by multiplying their relevance scores by their respective value vectors. With this step we get and improved and contextualized embedding representation of the current word.

Since each output y_i is computed independently this entire process can be parallelized by packing the input embeddings of the N tokens of the input sequence into a single matrix $X \in \mathbb{R}^{Nxd}$. Each row of X is the embedding of one token of the input. The next step is to multiply X by the key, query and value matrices

- $Q = XW^Q$
- $K = XW^K$
- $V = XW^V$

We can also get query and key together by multiplying Q and K^t. After we can scale these scores, take the softmax and then multiply the result by V. We can reduce self-attention as

$$\text{SelfAttention(Q, K, V)} = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

4.5.5 Deeper into the encoder block

4.5.6 The architecture of decoders